

@azdocuments

INTRODUCTION TO DBMS

SUBJECT CODE: BCS755A

MODULE-4

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

INTRODUCTION TO DBMS

BCS755A

MODULE-4

SQL: SQL data definition and data types, Schema change statements in SQL, specifying constraints in SQL, retrieval queries in SQL, INSERT, DELETE, and UPDATE statements in SQL, Additional features of SQL

Normalization: Database Design Theory – Introduction to Normalization using Functional and Multivalued Dependencies: Informal design guidelines for relation schema, Functional Dependencies, Normal Forms based on Primary Keys, Second and Third Normal Forms, Boyce-Codd Normal Form, Multivalued Dependency and Fourth Normal Form, Join Dependencies and Fifth Normal Form.

Textbook 1: Ch 6.1 to 6.5,14.1 to 14.7

4.1 SQL Data Definition and Data Types

Schema and Catalog Concepts in SQL

Introduction

SQL uses its own terminology, which corresponds directly to the formal relational model terms introduced in earlier chapters: SQL uses **table**, **row**, and **column** in place of the formal terms **relation**, **tuple**, and **attribute**, respectively. These corresponding terms are used interchangeably throughout the study of SQL. The main SQL command used for **data definition** is the **CREATE statement**, which can be used to create schemas, tables (relations), types, and domains, as well as other database constructs such as views, assertions, and triggers.

Before examining how tables themselves are created, it is important to first understand the concepts of **schema** and **catalog**, since these provide the organizational context within which tables and other database constructs actually exist.

The Concept of an SQL Schema

Early versions of SQL did **not** include the concept of a relational database schema at all in those versions, **all tables (relations) were considered part of the same, single schema**. The concept of an **SQL schema** was formally incorporated starting with **SQL2**, specifically in order to **group together tables and other constructs that belong to the same database application** (in some database systems, a schema is instead referred to as a **database**).

Structure of an SQL Schema

An **SQL schema** is identified by a **schema name**, and it includes:

- An **authorization identifier**, indicating the **user or account who owns** the schema.
- **Descriptors for each element** contained within the schema.

Schema elements may include **tables, types, constraints, views, domains**, and other constructs — such as **authorization grants** — that together describe the schema.

Creating a Schema

A schema is created using the **CREATE SCHEMA** statement, which can either:

1. **Include all the schema elements' definitions directly** within the same statement, **or**
2. **Assign only a name and authorization identifier** to the schema, with the individual elements (tables, etc.) **defined later** through separate statements.

Textbook Example:

CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';

Explanation: This statement creates a schema named **COMPANY**, owned by the user with the authorization identifier **'Jsmith'**. Note that, as with all SQL statements, this statement **ends with a semicolon**.

Important Note: Not all users are automatically authorized to create schemas and schema elements. The **privilege to create schemas, tables, and other constructs** must be **explicitly granted** to the relevant user accounts by the **system administrator or DBA (Database Administrator)**.

The Concept of a Catalog

In addition to the schema concept, SQL also uses the concept of a **catalog** — defined as a **named collection of schemas**. Database installations typically provide a **default environment and schema**, so that when a user connects and logs into that database installation, the user can refer directly to tables and other constructs within that default schema **without needing to explicitly specify a schema name** each time.

The INFORMATION_SCHEMA

A catalog **always contains a special schema** called **INFORMATION_SCHEMA**, which provides **information on all the schemas** contained within that catalog, as well as **all the element descriptors** present in those schemas. This special schema effectively serves as a built-in metadata repository describing the catalog's own structure.

Constraints and Sharing Across Schemas Within a Catalog

Integrity constraints, such as **referential integrity**, can be defined **between relations only if those relations exist in schemas within the same catalog** — referential integrity cannot span across entirely separate catalogs. Additionally, **schemas within the same catalog** can also **share certain elements**, such as **type and domain definitions**, enabling reuse of these definitions across multiple schemas belonging to the same catalog.

Important Notes

- SQL's terminology maps directly onto the relational model: **table = relation, row = tuple, column = attribute**.
- The **CREATE statement** is the primary SQL command for data definition — used for schemas, tables, types, domains, views, assertions, and triggers.
- **SQL schemas** were introduced starting with **SQL2**, to group related tables and constructs belonging to the same application.
- A schema has a **name**, an **authorization identifier** (owner), and **descriptors for its elements**.
- Schema creation privileges must be **explicitly granted** by the DBA/system administrator not all users can create schemas by default.
- A **catalog** is a **named collection of schemas**.
- Every catalog contains a special **INFORMATION_SCHEMA**, describing all schemas and elements within that catalog.

- **Referential integrity constraints require both relations to exist within the same catalog**.
- Schemas within the same catalog can **share type and domain definitions**.

The concepts of schema and catalog provide the essential organizational framework within which SQL tables and other database constructs are defined and managed. A schema groups together the tables, types, views, domains, and constraints belonging to a single database application, identified by a schema name and owned by a specific authorization identifier, while a catalog serves as a higher-level named collection of related schemas, complete with a built-in INFORMATION_SCHEMA describing its own contents. Understanding these two layers of organization — and the restriction that referential integrity constraints can only be specified between relations within the same catalog is foundational to correctly interpreting how the CREATE TABLE statements discussed in the next part are actually structured and scoped within a real SQL database system.

4.1.1 The CREATE TABLE Command in SQL

Introduction

Having established the organizational context provided by schemas and catalogs, this part focuses on the **core SQL command used to actually define a relation**: the **CREATE TABLE** command. This is arguably the single most important and most frequently tested piece of SQL DDL syntax in the VTU DBMS syllabus, since it directly implements the relational model concepts studied earlier — attributes, domains, primary keys, and referential integrity — as real, executable SQL statements. This part works through the complete COMPANY schema definition as the running textbook example.

Purpose of CREATE TABLE

The **CREATE TABLE** command is used to **specify a new relation**, by giving it a **name** and specifying its **attributes** and **initial constraints**. The attributes of the new table are specified **first**, and each attribute is given:

- A **name**,
- A **data type**, to specify its **domain of values**, and
- Possibly, **attribute constraints**, such as **NOT NULL**.

The **key, entity integrity, and referential integrity constraints** can be specified **within the CREATE TABLE statement itself**, immediately after the attributes are declared, **or** they can instead be **added later**, using the **ALTER TABLE** command.

Textbook Example The COMPANY Schema

Figure 6.1 in the textbook presents the complete set of CREATE TABLE statements needed to define the **COMPANY relational database schema**. The key statements are reproduced and explained below.

EMPLOYEE Table:

```
CREATE TABLE EMPLOYEE
( Fname       VARCHAR(15)     NOT NULL,
  Minit       CHAR,
  Lname       VARCHAR(15)     NOT NULL,
  Ssn         CHAR(9)         NOT NULL,
  Bdate       DATE,
  Address     VARCHAR(30),
  Sex         CHAR,
  Salary      DECIMAL(10,2),
  Super_ssn   CHAR(9),
  Dno         INT             NOT NULL,
  PRIMARY KEY (Ssn),
  FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),
  FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber) );
```

DEPARTMENT Table:

```
CREATE TABLE DEPARTMENT
( Dname           VARCHAR(15)   NOT NULL,
  Dnumber         INT           NOT NULL,
```

```
  Mgr_ssn         CHAR(9)      NOT NULL,
  Mgr_start_date  DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
```

DEPT_LOCATIONS Table:

```
CREATE TABLE DEPT_LOCATIONS
( Dnumber    INT          NOT NULL,
  Dlocation  VARCHAR(15)  NOT NULL,
  PRIMARY KEY (Dnumber, Dlocation),
  FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

PROJECT Table:

```
CREATE TABLE PROJECT
( Pname      VARCHAR(15)  NOT NULL,
  Pnumber    INT          NOT NULL,
  Plocation  VARCHAR(15),
  Dnum       INT          NOT NULL,
  PRIMARY KEY (Pnumber),
  UNIQUE (Pname),
  FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );
```

WORKS_ON Table:

```
CREATE TABLE WORKS_ON
( Essn    CHAR(9)        NOT NULL,
  Pno     INT            NOT NULL,
  Hours   DECIMAL(3,1)   NOT NULL,
```

```
  PRIMARY KEY (Essn, Pno),
  FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
  FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );
```

DEPENDENT Table:

```
CREATE TABLE DEPENDENT
( Essn            CHAR(9)       NOT NULL,
  Dependent_name   VARCHAR(15)   NOT NULL,
  Sex             CHAR,
  Bdate           DATE,
  Relationship     VARCHAR(8),
  PRIMARY KEY (Essn, Dependent_name),
  FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );
```

Explanation: Each CREATE TABLE statement lists the table's attributes along with their data types and NOT NULL constraints where applicable, followed by the key constraints (**PRIMARY KEY**, **UNIQUE**) and the **FOREIGN KEY … REFERENCES** clauses, which directly implement the referential integrity constraints studied earlier — for example, EMPLOYEE.Dno referencing DEPARTMENT.Dnumber, and EMPLOYEE.Super_ssn referencing EMPLOYEE.Ssn itself (the self-referencing foreign key discussed in Chapter 5).

Specifying the Schema Explicitly

Typically, the **SQL schema** in which relations are declared is **implicitly specified** by the environment in which the CREATE TABLE statements are executed. Alternatively, the **schema name can be explicitly attached** to the relation name, separated by a **period (.)**. For example, writing:

CREATE TABLE COMPANY.EMPLOYEE

rather than simply:

CREATE TABLE EMPLOYEE

makes the EMPLOYEE table **explicitly** (rather than implicitly) part of the **COMPANY** schema.

Base Tables vs. Virtual Relations

The relations declared through CREATE TABLE statements are called **base tables** (or **base relations**) — this terminology signifies that the table and its rows are **actually created and stored as a file** by the DBMS. Base relations are distinguished from **virtual relations**, which are instead created through the **CREATE VIEW** statement, and which **may or may not correspond to an actual physical file**.

Ordering in Base Tables

In SQL, the **attributes in a base table are considered to be ordered**, in the exact **sequence in which they are specified** in the CREATE TABLE statement. However, consistent with the formal relational model, the **rows (tuples) of the table are NOT considered to be ordered** within the table.

The Circular Reference and Forward Reference Problem

An important, often-tested practical issue arises in the COMPANY schema example: **some foreign keys may cause errors** when the tables are created in a straightforward top-to-bottom order, because they are specified either via:

- **Circular references** — where a table's foreign key refers back to **itself**. For example, the foreign key **Super_ssn** in the EMPLOYEE table is a circular reference, since it refers to the **EMPLOYEE table itself**.
- **Forward references** — where a table's foreign key refers to **another table that has not yet been created**. For example, the foreign key **Dno** in the EMPLOYEE table refers to the **DEPARTMENT table**, which, in the order shown, **has not been created yet** at the point EMPLOYEE is being defined.

Solution: To deal with this type of problem, such constraints can simply be **left out of the initial CREATE TABLE statement**, and then **added later** using the **ALTER TABLE** statement, once all the referenced tables already exist.

Important Note: The textbook displays **all** the foreign keys directly within the initial CREATE TABLE statements (as shown above) purely in order to present the **complete COMPANY schema in one place** for clarity — in actual practice, some of these foreign

key constraints would need to be added afterward via ALTER TABLE to avoid the circular/forward reference problem.

- CREATE TABLE specifies a **name, attributes (with data types), and constraints** for a new relation.
- Key and referential integrity constraints can be specified **within** CREATE TABLE, or added **later** via **ALTER TABLE**.
- A relation's schema can be **explicitly qualified**, e.g., **COMPANY.EMPLOYEE**, using a period to separate schema name and table name.
- Relations created via CREATE TABLE are called **base tables/relations** — physically stored by the DBMS — distinguished from **virtual relations** (views), created via CREATE VIEW.
- **Attributes are ordered** in a base table (per their declaration sequence); **rows are NOT ordered**.
- **Circular references** (e.g., Super_ssn → EMPLOYEE itself) and **forward references** (e.g., Dno → DEPARTMENT, not yet created) can cause errors if declared directly during table creation — such constraints are typically added later via **ALTER TABLE**.

The CREATE TABLE command serves as the practical, executable SQL realization of the relational model concepts studied throughout Chapter 5 — attributes and their domains become typed columns, primary keys and NOT NULL constraints implement entity integrity, and FOREIGN KEY … REFERENCES clauses implement referential integrity. The complete COMPANY schema example demonstrates not only the correct syntax for defining six interrelated tables, but also an important practical caveat: circular and forward-referencing foreign keys, such as EMPLOYEE's self-referencing Super_ssn or its forward-referencing Dno, may require their constraints to be added after table creation using ALTER TABLE, rather than being declared inline within the initial CREATE TABLE statement.

4.1.2 Attribute Data Types and Domains in SQL

Introduction

Every attribute declared within a CREATE TABLE statement must be assigned a **data type**, which formally specifies the **domain of values** that attribute is permitted to hold directly implementing the domain constraint concept studied in Chapter 5. SQL provides a rich collection of built-in data types, spanning numeric, character string, bit string, Boolean, date, and time categories, along with several more specialized types for large objects and time intervals. This part works through each of these categories in detail, along with SQL's support for user-defined **domains**.

Overview of Basic Data Type Categories

The basic data types available for attributes in SQL include: **numeric, character string, bit string, Boolean, date, and time**.

Numeric Data Types

Numeric data types include:

- **Integer numbers** of various sizes — **INTEGER** (or **INT**) and **SMALLINT**.
- **Floating-point (real) numbers** of various precision — **FLOAT** (or **REAL**) and **DOUBLE PRECISION**.

Formatted numbers can be declared using **DECIMAL(i, j)** — or equivalently **DEC(i, j)** or **NUMERIC(i, j)** — where:

- **i**, the **precision**, is the **total number of decimal digits**.
- **j**, the **scale**, is the **number of digits after the decimal point**.

The **default for scale is zero**, and the **default for precision is implementation-defined** (i.e., it depends on the specific DBMS being used).

Character-String Data Types

Character-string data types come in two forms:

- **Fixed length — CHAR(n)** or **CHARACTER(n)**, where **n** is the fixed number of characters.
- **Varying length — VARCHAR(n)**, **CHAR VARYING(n)**, or **CHARACTER VARYING(n)**, where **n** is the **maximum** number of characters.

Literal string values are placed between **single quotation marks (apostrophes)**, and string comparison in SQL is **case sensitive** — a distinction is made between uppercase and lowercase letters. (This case sensitivity does **not** apply to SQL **keywords**, such as CREATE or CHAR — SQL is **case insensitive with respect to its own keywords**.)

Padding of Fixed-Length Strings: For fixed-length strings, a **shorter string is padded with blank characters to the right** to reach the declared length. For example, the value 'Smith' stored in an attribute of type CHAR(10) is padded with **five blank characters** to fill the declared width. **Padded blanks are generally ignored** when strings are compared for equality.

String Ordering: For comparison purposes, strings are considered ordered in **alphabetic (lexicographic) order** — if string str1 appears before string str2 in alphabetic order, then str1 is considered **less than** str2. (Non-alphabetic characters also have a defined order.)

Concatenation: SQL provides a **concatenation operator**, denoted by || **(double vertical bar)**, which concatenates two strings together. For example, **'abc' || 'XYZ'** results in the single string **'abcXYZ'**.

CLOB — Character Large Object: A variable-length string data type called **CHARACTER LARGE OBJECT (CLOB)** is available for columns holding **large text values**, such as full documents. The **CLOB maximum length** can be specified in **kilobytes (K), megabytes (M), or gigabytes (G)** — for example, **CLOB(20M)** specifies a maximum length of 20 megabytes.

Bit-String Data Types

Bit-string data types similarly come in two forms:

- **Fixed length n — BIT(n)**.
- **Varying length — BIT VARYING(n)**, where **n** is the **maximum** number of bits.

The **default length** for both character strings and bit strings, if unspecified, is **1**.

Literal bit strings are placed between single quotes, but are **preceded by the letter B** to distinguish them from ordinary character strings — for example, **B'10101'**. (Bit strings whose length is a multiple of 4 can alternatively be specified in **hexadecimal notation**, preceded by **X**, where each hexadecimal character represents 4 bits.)

BLOB — Binary Large Object: A variable-length bit-string data type called **BINARY LARGE OBJECT (BLOB)** is available for columns holding **large binary values**, such as images. As with CLOB, the maximum length of a BLOB can be specified in **kilobits (K), megabits (M), or gigabits (G)** — for example, **BLOB(30G)** specifies a maximum length of 30 gigabits.

Boolean Data Type

The **Boolean** data type has the traditional values of **TRUE** or **FALSE**. However, because of the presence of **NULL values** in SQL, a **three-valued logic** is actually used — meaning a **third possible value, UNKNOWN**, also exists for a Boolean data type (this three-valued logic is discussed in greater detail in Chapter 7).

DATE and TIME Data Types

DATE — has **ten positions**, with components **YEAR, MONTH, and DAY**, in the format **YYYY-MM-DD**.

TIME — has **at least eight positions**, with components **HOUR, MINUTE, and SECOND**, in the format **HH:MM:SS**.

Validity Requirement: Only **valid** dates and times should be permitted by the SQL implementation — this means months must be between **1 and 12**, days must be between **01 and 31**, and additionally, a given day must be a **valid day for its corresponding month** (e.g., no February 30).

Comparison: The **< (less than)** comparison operator can be applied to dates or times — an **earlier date is considered smaller** than a later date, and similarly for time values.

Literal Representation: Literal date and time values are represented as **single-quoted strings preceded by the keyword DATE or TIME** — for example, **DATE '2014-09-27'** or **TIME '09:12:47'**.

TIME(i): A data type **TIME(i)**, where **i** is called the **time fractional seconds precision**, specifies **i + 1 additional positions** for TIME — one position for an additional period (.) separator character, and **i** positions for specifying decimal fractions of a second.

TIME WITH TIME ZONE: This variant includes **an additional six positions** for specifying the **displacement from the standard universal time zone**, ranging from **+13:00 to −12:59**, in units of **HOURS:MINUTES**. If **WITH TIME ZONE** is **not** included, the **default is the local time zone** of the current SQL session.

Additional Data Types

TIMESTAMP: Includes both the **DATE and TIME fields**, plus a **minimum of six positions for decimal fractions of seconds**, and an **optional WITH TIME ZONE qualifier**. Literal values are represented as single-quoted strings preceded by the keyword **TIMESTAMP**, with a **blank space** separating the date and time portions — for example, **TIMESTAMP '2014-09-27 09:12:47.648302'**.

INTERVAL: A data type related to DATE, TIME, and TIMESTAMP, used to specify an **interval** — a **relative value** that can be used to **increment or decrement** an absolute date, time, or timestamp value. Intervals are qualified as either **YEAR/MONTH intervals** or **DAY/TIME intervals**.

Important Note — String Comparability: The formats of DATE, TIME, and TIMESTAMP can generally be treated as a **special type of string**. Consequently, they can typically be used directly in **string comparisons**, by being **cast (coerced, or converted)** into their equivalent string representations.

Domains in SQL

Rather than specifying the data type of every attribute directly (as shown in the earlier CREATE TABLE examples), it is also possible to first **declare a domain**, and then simply use that **domain name** wherever the corresponding attribute is specified. This approach offers two key advantages:

1. It makes it **easier to change the data type** for a domain that happens to be **used by numerous attributes** across a schema — the change need only be made once, at the domain definition, rather than at every individual attribute.
2. It **improves schema readability**, since domain names can be chosen to be more descriptive than raw data type declarations.

Textbook Example:

CREATE DOMAIN SSN_TYPE AS CHAR(9);

Explanation: This statement creates a domain named **SSN_TYPE**, defined as CHAR(9). This domain name could then be used **in place of CHAR(9)** for every Ssn-related attribute across the COMPANY schema — namely, **Ssn** and **Super_ssn** of EMPLOYEE, **Mgr_ssn** of DEPARTMENT, and **Essn** of both WORKS_ON and

DEPENDENT — ensuring all these related attributes remain consistently typed, and easy to update together if the type ever needs to change.

- A domain can also have an **optional default specification**, via a **DEFAULT clause** (similar to how default values can be specified for ordinary attributes).
- **Domains may not be available in all implementations of SQL** — this is an implementation-dependent feature.

The CREATE TYPE Command

SQL also provides a **CREATE TYPE** command, which can be used to create **User Defined Types (UDTs)**. These user-defined types can then be used either as **data types for individual attributes**, or as the **basis for creating entire tables**. This feature is discussed in greater detail in later chapters, particularly in conjunction with **object database features** that have been incorporated into more recent versions of the SQL standard.

Important Notes

- **Data type categories**: numeric, character string, bit string, Boolean, date, time (plus TIMESTAMP and INTERVAL as extensions).
- **DECIMAL(i, j)**: i = precision (total digits), j = scale (digits after decimal point); default scale = 0.
- **CHAR(n)** = fixed length (padded with blanks); **VARCHAR(n)** = variable length (up to n).
- String comparison is **case sensitive**; SQL keywords are **case insensitive**.
- **CLOB** = large text objects (K/M/G); **BLOB** = large binary objects (K/M/G).
- SQL uses **three-valued logic** for Boolean: TRUE, FALSE, and **UNKNOWN** (due to NULLs).
- **DATE** format: YYYY-MM-DD (10 positions); **TIME** format: HH:MM:SS (≥8 positions).
- **TIMESTAMP** = DATE + TIME + fractional seconds (+ optional time zone).
- **INTERVAL** = relative value to increment/decrement a date/time/timestamp; either YEAR/MONTH or DAY/TIME type.

- **Domains** (CREATE DOMAIN) allow a data type to be named and reused across multiple attributes — easing maintenance and improving readability; **not available in all SQL implementations**.
- **CREATE TYPE** defines **User Defined Types (UDTs)**, usable as attribute types or as a basis for tables.

SQL's data type system provides a comprehensive set of built-in types numeric, character string, bit string, Boolean, date, time, and their extensions such as TIMESTAMP, INTERVAL, CLOB, and BLOB that together allow precise, domain-constraint-enforcing specification of every attribute declared in a CREATE TABLE statement. Beyond directly specifying data types attribute-by-attribute, SQL's CREATE DOMAIN command offers a more maintainable alternative, allowing a single named domain, such as SSN_TYPE, to be reused consistently across multiple related attributes throughout a schema. Together with the more advanced CREATE TYPE command for user-defined types, these features give SQL considerable flexibility in accurately and efficiently modeling the domains that underlie a relational database schema.

4.2 Specifying Constraints in SQL

4.2.1 Specifying Attribute Constraints and Attribute Defaults

Introduction

This section describes the **basic constraints** that can be specified in SQL as part of table creation including **key and referential integrity constraints**, restrictions on **attribute domains and NULLs**, and constraints on **individual tuples** using the **CHECK clause**. This part specifically focuses on **attribute-level constraints**: the NOT NULL constraint, default values for attributes, and domain-restricting CHECK clauses.

The NOT NULL Constraint

Because SQL **allows NULLs** as attribute values by default, a constraint called **NOT NULL** may be specified if **NULL is not permitted** for a particular attribute. This constraint is **always implicitly specified** for attributes that are part of the **primary key** of a relation (directly implementing the entity integrity constraint from Chapter 5) — but it can also be **explicitly specified** for any other attributes whose values are required to never be NULL.

Default Values for Attributes

It is also possible to define a **default value** for an attribute, by appending the clause **DEFAULT <value>** to that attribute's definition. The **default value is included** in any new tuple whenever an **explicit value is not provided** for that attribute during insertion.

Textbook Examples — Default Values:

```
CREATE TABLE EMPLOYEE
( … ,
  Dno INT NOT NULL DEFAULT 1,
  … );
CREATE TABLE DEPARTMENT
( … ,
  Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555',
  … );
```

Explanation: In the EMPLOYEE table, if a new employee tuple is inserted **without** specifying a value for Dno, the DBMS will automatically assign it the **default value of 1**. Similarly, in the DEPARTMENT table, if Mgr_ssn is left unspecified during insertion, it will default to **'888665555'** — representing a default manager for a newly created department.

Important Note: If **no DEFAULT clause** is specified for an attribute, the **default value is NULL**, provided that attribute does **not** carry a NOT NULL constraint.

The CHECK Clause for Attribute/Domain Restrictions

Another type of constraint can **restrict the permissible values** of an attribute or domain, using the **CHECK clause**, placed following an attribute or domain definition.

Textbook Example — CHECK on an Attribute:

Suppose department numbers are restricted to integer values **between 1 and 20**. The Dnumber attribute declaration in DEPARTMENT could then be written as:

Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);

Explanation: This ensures that any attempt to insert or update a Dnumber value **outside the range 1–20** will be **rejected** by the DBMS, directly enforcing a domain constraint at the database level.

CHECK Used with CREATE DOMAIN

The CHECK clause can also be used **in conjunction with the CREATE DOMAIN statement**, allowing the same restriction to be defined once and reused across multiple attributes.

Textbook Example:

CREATE DOMAIN D_NUM AS INTEGER

CHECK (D_NUM > 0 AND D_NUM < 21);

Explanation: Once this domain **D_NUM** is created, it can be used as the **attribute type** for **all attributes that refer to department numbers** throughout the schema — such as **Dnumber** of DEPARTMENT, **Dnum** of PROJECT, and **Dno** of EMPLOYEE — ensuring the same range restriction is **consistently enforced** everywhere the concept of "department number" appears, without needing to repeat the CHECK condition in every individual table definition.

- **NOT NULL** is **automatically implied** for primary key attributes; it can be **explicitly added** to any other attribute as needed.
- **DEFAULT <value>** supplies a value automatically when an attribute is **omitted** during an INSERT.
- If **no DEFAULT** is specified and the attribute is **not NOT NULL**, the implicit default is **NULL**.
- **CHECK (condition)** restricts the set of valid values an attribute (or domain) may take — directly enforcing **domain constraints** at the SQL level.
- CHECK can be applied either to an **individual attribute** within CREATE TABLE, or to a **domain** within CREATE DOMAIN, the latter allowing reuse across multiple attributes.

SQL provides three complementary mechanisms for enforcing attribute-level constraints: **NOT NULL**, which prevents missing values where they are not permitted; **DEFAULT**, which automatically supplies a sensible value when none is explicitly provided; and **CHECK**, which restricts an attribute (or an entire reusable domain) to

only a specified range or set of valid values. Together, these mechanisms allow the abstract domain and NULL constraints studied in the formal relational model to be directly and practically enforced within a real SQL schema, as demonstrated by the EMPLOYEE.Dno default, the DEPARTMENT.Mgr_ssn default, and the department-number range restriction expressed through both an inline CHECK and a reusable D_NUM domain.

4.2.2 Specifying Key and Referential Integrity Constraints

Because keys and referential integrity constraints are considered **very important**, SQL provides **special clauses** within the CREATE TABLE statement specifically to specify them.

The PRIMARY KEY Clause

The **PRIMARY KEY** clause specifies **one or more attributes** that make up the primary key of a relation. If the primary key consists of a **single attribute**, the clause can follow that attribute **directly**, rather than being listed separately at the end of the statement.

Textbook Example:

Dnumber INT PRIMARY KEY,

Explanation: This is an alternative, more compact way of declaring Dnumber as the primary key of DEPARTMENT, instead of listing PRIMARY KEY(Dnumber) as a separate clause at the end of the CREATE TABLE statement.

The UNIQUE Clause

The **UNIQUE** clause specifies **alternate (unique) keys**, also known as **candidate keys**. Like PRIMARY KEY, the UNIQUE clause can also be specified **directly** for a single-attribute unique key.

Textbook Example:

Dname VARCHAR(15) UNIQUE,

Explanation: This declares Dname as a candidate key of DEPARTMENT — ensuring no two departments can share the same name — without designating it as the primary key.

The FOREIGN KEY Clause and Referential Integrity Violations

Referential integrity is specified via the **FOREIGN KEY** clause. As discussed earlier (Section 5.2.4 and 5.3), a referential integrity constraint can be **violated** when tuples are **inserted or deleted**, or when a **foreign key or primary key attribute value is updated**.

Default Action — RESTRICT: The **default action** that SQL takes for an integrity violation is to **reject the update operation** that would cause the violation — this default behavior is known as the **RESTRICT** option.

Referential Triggered Actions

The **schema designer can specify an alternative action** to be taken, instead of the default RESTRICT, by attaching a **referential triggered action clause** to any foreign key constraint. The available options are:

- **SET NULL**
- **CASCADE**
- **SET DEFAULT**

Each such option **must be qualified** with either **ON DELETE** or **ON UPDATE**, since the appropriate action may differ depending on whether the violation was triggered by a deletion or an update.

Textbook Example — EMPLOYEE.Super_ssn:

CONSTRAINT EMPSUPERFK

FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)

ON DELETE SET NULL ON UPDATE CASCADE,

Explanation: Here, the database designer has chosen **ON DELETE SET NULL** and **ON UPDATE CASCADE** for the foreign key Super_ssn of EMPLOYEE. This means:

- If the tuple for a **supervising employee is deleted**, the value of **Super_ssn is automatically set to NULL** for all employee tuples that were referencing that deleted employee.
- If the **Ssn value of a supervising employee is updated** (for example, because it was originally entered incorrectly), the **new value is automatically cascaded** to the Super_ssn attribute of all employee tuples that were referencing the updated employee tuple.

Textbook Example — EMPLOYEE.Dno:

CONSTRAINT EMPDEPTFK

FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)

ON DELETE SET DEFAULT ON UPDATE CASCADE);

Explanation: Here, if the referenced DEPARTMENT tuple is deleted, Dno is **set to its DEFAULT value** (recall Dno defaults to 1) for all employees who were in that department; if the department's Dnumber is updated, the change **cascades** to Dno for all affected employees.

General Behavior of the Referential Triggered Actions

- **SET NULL** (for both ON DELETE and ON UPDATE): The value of the affected **referencing attribute(s) is changed to NULL**.
- **SET DEFAULT** (for both ON DELETE and ON UPDATE): The value of the affected referencing attribute(s) is changed to the **specified default value** of that attribute.
- **CASCADE ON DELETE**: **Deletes all the referencing tuples** as well — propagating the deletion.
- **CASCADE ON UPDATE**: **Changes the value** of the referencing foreign key attribute(s) to the **updated (new) primary key value**, for all referencing tuples.

Important Note — Responsibility of the Designer: It is the **responsibility of the database designer** to choose the appropriate action for each foreign key, and to specify it explicitly in the database schema. As a **general rule**:

- The **CASCADE option** is suitable for **"relationship" relations** (such as **WORKS_ON**), for relations representing **multivalued attributes** (such as **DEPT_LOCATIONS**), and for relations representing **weak entity types** (such as **DEPENDENT**) — in all these cases, the referencing tuples exist *only* to describe the referenced entity, so it makes sense for them to be deleted or updated along with it.

4.2.3 Giving Names to Constraints

A constraint may optionally be given a **constraint name**, by preceding its definition with the keyword **CONSTRAINT**, followed by the chosen name.

Textbook Example:

CONSTRAINT EMPPK

PRIMARY KEY (Ssn),

Rules and Purpose of Constraint Names:

- The **names of all constraints within a particular schema must be unique**.
- A constraint name is useful for **identifying a particular constraint later**, especially if that constraint needs to be **dropped and replaced** with another one (discussed further under ALTER TABLE in Chapter 7).
- **Giving names to constraints is optional** — an unnamed constraint is still fully valid and enforced, but cannot as easily be individually referenced later.
- It is also possible to **temporarily defer** a constraint until the end of a transaction (discussed further in the context of transaction concepts).

4.2.4 Specifying Constraints on Tuples Using CHECK

In addition to key and referential integrity constraints (specified via dedicated keywords), **other table-level constraints** can be specified through **additional CHECK clauses** placed at the **end** of a CREATE TABLE statement. These are called **row-based constraints**, because they apply to **each row individually**, and are checked **whenever a row is inserted or modified**.

Textbook Example:

Suppose the DEPARTMENT table had an additional attribute **Dept_create_date**, storing the date the department was created. A CHECK clause could then be added at the end of the CREATE TABLE statement to ensure that a manager's start date is **later than** the department's creation date:

CHECK (Dept_create_date <= Mgr_start_date);

Explanation: Unlike the earlier CHECK examples, which restricted a **single** attribute's values independently, this row-based CHECK clause compares **two different attributes within the same row** against each other, ensuring a logical relationship holds between them for every tuple in the table.

Important Note: The CHECK clause can also be used to specify **more general constraints** through the **CREATE ASSERTION** statement of SQL — this is discussed

further in the context of more advanced queries, since it requires the full power of SQL's querying capabilities.

Important Notes

- **PRIMARY KEY** and **UNIQUE** clauses can be written inline (for single-attribute keys) or as separate clauses (for composite keys).
- **FOREIGN KEY … REFERENCES** specifies referential integrity; the **default violation-handling action is RESTRICT** (reject the operation).
- **Referential triggered actions — SET NULL, CASCADE, SET DEFAULT** — must each be qualified with **ON DELETE** or **ON UPDATE**, and can differ between the two.
- **CASCADE** is generally appropriate for **relationship relations, multivalued-attribute relations, and weak entity relations**.
- Constraints can be **explicitly named** using **CONSTRAINT <name>**, which must be **unique within the schema**; naming is **optional** but useful for later modification.
- **Row-based CHECK clauses**, placed at the end of CREATE TABLE, enforce conditions across **multiple attributes within the same tuple**, checked on every insert/update.

Conclusion

SQL provides a comprehensive and flexible set of tools for specifying key and referential integrity constraints directly within the CREATE TABLE statement: PRIMARY KEY and UNIQUE clauses implement the primary and candidate key concepts, while FOREIGN KEY … REFERENCES, combined with configurable ON DELETE and ON UPDATE referential triggered actions (RESTRICT by default, or SET NULL, CASCADE, SET DEFAULT as alternatives), give the database designer precise control over how referential integrity violations are automatically resolved — as illustrated by the EMPLOYEE.Super_ssn (SET NULL / CASCADE) and EMPLOYEE.Dno (SET DEFAULT / CASCADE) examples. Naming constraints via the CONSTRAINT keyword further supports schema maintainability, while row-based CHECK clauses extend validation beyond single attributes to enforce logical relationships across an entire tuple, such as ensuring a manager's start date follows the department's creation date.

Chapter 4.3 – Basic Retrieval Queries in SQL

4.3.1 The SELECT-FROM-WHERE Structure of Basic SQL Queries

Introduction

Having covered how tables and their constraints are defined, we now turn to **retrieving data** from a relational database — the single most heavily used and heavily examined capability of SQL. SQL provides **one basic statement** for retrieving information: the **SELECT statement**. This part introduces the fundamental **SELECT-FROM-WHERE** structure, works through several classic textbook example queries, and highlights an important conceptual distinction between SQL tables and the formal relational model.

SQL SELECT vs. Relational Algebra SELECT

Important Note: The SQL **SELECT statement** is **not the same** as the **SELECT operation** of relational algebra (discussed in Chapter 8). While relational algebra's SELECT corresponds to *filtering rows*, SQL's SELECT statement is a **much more general retrieval construct**, capable of performing selection, projection, and joining all within a single statement. There are many options and variations to the SQL SELECT statement, introduced gradually throughout this and later chapters.

SQL Tables: Multisets, Not Sets

Before working through example queries, it is important to highlight a key distinction between the **practical SQL model** and the **formal relational model** discussed in Chapter 5: **SQL allows a table to have two or more tuples that are identical in all their attribute values**. This means that, in general, an **SQL table is NOT a set of tuples** — because a mathematical set does not permit two identical members — but is instead a **multiset** (sometimes called a **bag**) of tuples.

Important Note: Some SQL relations **are** constrained to behave as true sets — this happens either because a **key constraint** has been declared on the relation (guaranteeing no two tuples can be fully identical), or because the **DISTINCT** option has been explicitly used with a SELECT statement.

The SELECT-FROM-WHERE Block

The basic form of the SELECT statement — sometimes called a **mapping**, or a **select-from-where block** — is composed of **three clauses**: **SELECT, FROM, and WHERE**, structured as follows:

SELECT <attribute list>

FROM <table list>

WHERE <condition>;

Where:

- **<attribute list>** — a list of attribute names whose values are to be **retrieved** by the query.
- **<table list>** — a list of the **relation names** required to process the query.
- **<condition>** — a **conditional (Boolean) expression** that identifies the tuples to be retrieved by the query.

Important Note: The **SELECT and FROM clauses are required** in all SQL queries; the **WHERE clause is optional**.

Comparison Operators

In SQL, the basic **logical comparison operators** for comparing attribute values with each other, or with literal constants, are:

=, <, <=, >, >=, and <>

These correspond directly to the relational algebra operators **=, <, ≤, >, ≥, and ≠** respectively, and also correspond to the familiar C/C++ programming language operators **=, <, <=, >, >=, and !=**. The **main syntactic difference** is the **"not equal"** operator, which SQL expresses as **<>**.

Textbook Example — Query 0

Query: Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

Q0: SELECT Bdate, Address

FROM EMPLOYEE

WHERE Fname = 'John' AND Minit = 'B' AND Lname = 'Smith';

Explanation: This query involves only the **EMPLOYEE** relation listed in the FROM clause. It **selects** the individual EMPLOYEE tuples satisfying the WHERE condition, and then **projects** the result onto just the **Bdate** and **Address** attributes listed in the SELECT clause.

Correspondence to Relational Algebra Terminology

- The **SELECT clause** of SQL specifies the attributes whose values are to be retrieved — these are called the **projection attributes** in relational algebra.
- The **WHERE clause** specifies the Boolean condition that must be true for any retrieved tuple — this is known as the **selection condition** in relational algebra.

Conceptual Model — Implicit Tuple Variable: We can think of an **implicit tuple variable (or iterator)** in the SQL query, conceptually ranging or looping over **each individual tuple** in the EMPLOYEE table, and **evaluating the WHERE condition** for that tuple. Only those tuples for which the condition evaluates to **TRUE**, after substituting the tuple's actual attribute values, are **selected** for the result.

Textbook Example — Query 1 (Select-Project-Join)

Query: Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1: SELECT Fname, Lname, Address
    FROM EMPLOYEE, DEPARTMENT
    WHERE Dname = 'Research' AND Dnumber = Dno;
```

Explanation of the WHERE Clause's Two Conditions:

- **Dname = 'Research'** is a **selection condition** — it chooses the particular tuple of interest within the DEPARTMENT table, since Dname is an attribute of DEPARTMENT.
- **Dnumber = Dno** is called a **join condition** — it combines two tuples, one from DEPARTMENT and one from EMPLOYEE, whenever the Dnumber value in DEPARTMENT matches the Dno value in EMPLOYEE.

Important Note: In general, **any number of selection and join conditions** may be specified together within a single SQL query. A query that involves only **selection**

conditions, join conditions, and projection attributes is known as a **select-project-join query**.

Textbook Example — Query 2 (Two Join Conditions)

Query: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
Q2: SELECT Pnumber, Dnum, Lname, Address, Bdate
    FROM PROJECT, DEPARTMENT, EMPLOYEE
    WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND
        Plocation = 'Stafford';
```

Explanation: This query involves **two join conditions**:

- **Dnum = Dnumber** relates a project tuple to its **controlling department** tuple.
- **Mgr_ssn = Ssn** relates that controlling department tuple to the **employee tuple who manages it**.

Each tuple in the final result is thus a combination of **one project, one department that controls it, and one employee who manages that department**. The projection attributes listed in the SELECT clause then determine which specific attributes from this combined tuple are actually displayed in the output.

Important Notes

- SQL's **SELECT statement** ≠ relational algebra's SELECT **operation** — SQL's SELECT is far more general.
- SQL tables are, in general, **multisets (bags)**, not true sets — duplicates are allowed unless a **key constraint** or **DISTINCT** enforces uniqueness.
- The SELECT-FROM-WHERE block: **SELECT** = projection attributes, **FROM** = table list, **WHERE** = selection/join conditions (optional clause).
- SQL comparison operators: **=, <, <=, >, >=, <>** — note **<>** for "not equal" (vs. relational algebra's ≠).
- A **select-project-join query** combines selection conditions, join conditions, and projection attributes in one SELECT-FROM-WHERE statement.

- Multiple join conditions can be chained together in a single WHERE clause to combine more than two relations.

The SELECT-FROM-WHERE block forms the foundational structure of virtually every SQL retrieval query, directly mirroring — yet extending beyond — the SELECT, PROJECT, and JOIN operations of relational algebra within a single unified statement. As demonstrated by Query 0 (simple selection and projection), Query 1 (a select-project-join combining one selection condition and one join condition), and Query 2 (chaining two join conditions across three relations), the WHERE clause plays a dual role — expressing both selection conditions that filter individual relations and join conditions that combine tuples across multiple relations — making it the most conceptually rich clause of the basic SQL query structure.

4.3.2 Ambiguous Attribute Names, Aliasing, Renaming, and Tuple Variables

Introduction

As queries begin to involve **multiple tables**, a practical naming problem can arise: two different relations may happen to use the **same attribute name** for conceptually different things, or a query may need to reference the **same relation more than once** within a single statement. SQL addresses both situations through **attribute qualification** and the use of **aliases (tuple variables)** — mechanisms that are essential for writing correct, unambiguous, and readable multi-table queries.

Same Attribute Name Across Different Tables

In SQL, the **same name can be used for two (or more) attributes**, as long as those attributes belong to **different tables**. When this happens, and a **multitable query** refers to two or more attributes sharing the same name, we **must qualify** the attribute name with its relation name, to **prevent ambiguity**. This qualification is done by **prefixing the relation name to the attribute name**, separated by a **period**.

Textbook Example — Qualifying Ambiguous Attributes

Suppose, hypothetically, that the Dno and Lname attributes of EMPLOYEE were instead called **Dnumber** and **Name**, and the Dname attribute of DEPARTMENT was **also** called **Name**. Then, to prevent ambiguity, Query 1 would need to be rephrased as follows:

```
Q1A: SELECT Fname, EMPLOYEE.Name, Address
    FROM EMPLOYEE, DEPARTMENT
    WHERE DEPARTMENT.Name = 'Research' AND
        DEPARTMENT.Dnumber = EMPLOYEE.Dnumber;
```

Explanation: Here, the attributes **Name** and **Dnumber** must be **prefixed** with their respective relation names (EMPLOYEE.Name vs. DEPARTMENT.Name; DEPARTMENT.Dnumber vs. EMPLOYEE.Dnumber), since **both relations** use these exact same attribute names for different concepts — without this qualification, SQL would be unable to determine which relation's attribute is being referenced.

Fully Qualified Names for Clarity (Even Without Ambiguity)

Important Note: Fully qualified attribute names **can be used for clarity even when there is no actual ambiguity** in the attribute names. Query 1 can therefore be rewritten, using fully qualified names throughout, as **Q1′**:

```
Q1': SELECT EMPLOYEE.Fname, EMPLOYEE.LName, EMPLOYEE.Address
    FROM EMPLOYEE, DEPARTMENT
    WHERE DEPARTMENT.DName = 'Research' AND
        DEPARTMENT.Dnumber = EMPLOYEE.Dno;
```

Aliasing — Renaming Table Names

We can also **rename table names to shorter aliases**, avoiding the need to repeatedly type out long table names throughout a query. This becomes especially important — indeed, **necessary** — when a query needs to refer to the **same relation more than once**.

Textbook Example — Query 8: Self-Join Using Aliases

Query: For each employee, retrieve the employee's first and last name, and the first and last name of his or her immediate supervisor.

```
Q8: SELECT E.Fname, E.Lname, S.Fname, S.Lname
   FROM EMPLOYEE AS E, EMPLOYEE AS S
   WHERE E.Super_ssn = S.Ssn;
```

Explanation: Since this query needs to reference the **EMPLOYEE relation twice** — once in the role of the **supervisee/subordinate**, and once in the role of the **supervisor** — we are required to declare **alternative relation names**, **E** and **S**, called **aliases** or **tuple variables**, for EMPLOYEE.

Syntax for Aliases: An alias can be specified either by following the keyword **AS** (as shown in Q8), or by writing it **directly after** the relation name, without AS — for example, **EMPLOYEE E, EMPLOYEE S** in the FROM clause would work identically.

Conceptual Model: We can think of **E** and **S** as **two different (conceptual) copies** of the EMPLOYEE relation: **E** represents employees in the role of **supervisees/subordinates**, while **S** represents employees in the role of **supervisors**. We then **join these two copies** together. In reality, of course, there is only **one** EMPLOYEE relation — the join condition **E.Super_ssn = S.Ssn** is specifically designed to **join the relation with itself**, matching tuples that satisfy this self-referencing condition.

Important Note — Recursive Query: This is an example of a **one-level recursive query**. In **earlier versions of SQL**, it was **not possible** to specify a **general** recursive query — one with an **unknown number of levels** — within a single SQL statement. A dedicated construct for specifying such recursive queries was later **incorporated into SQL:1999**.

Renaming Attributes Using Aliases

It is also possible to **rename the attributes of a relation** within a query, by giving them aliases directly in the FROM clause. For example, writing:

EMPLOYEE AS E(Fn, Mi, Ln, Ssn, Bd, Addr, Sex, Sal, Sssn, Dno)

in the FROM clause makes **Fn** an alias for Fname, **Mi** an alias for Minit, **Ln** an alias for Lname, and so on, for the remaining attributes in order.

General Practice — Using Aliases Even Without Necessity

Whenever one or more aliases are assigned to a relation, those aliases can be used to represent **different references to that same relation**, permitting **multiple references to the same relation within a single query** — as required in Q8.

Important Note — Recommended Practice: This alias-naming (renaming) mechanism can be used in **any** SQL query, to specify tuple variables for **every table** in the WHERE clause — **whether or not** the same relation actually needs to be referenced

more than once. In fact, this practice is **recommended**, since it generally results in queries that are **easier to comprehend**.

Textbook Example — Query 1 Rewritten with Aliases (Q1B):

```
Q1B: SELECT E.Fname, E.LName, E.Address
     FROM EMPLOYEE AS E, DEPARTMENT AS D
     WHERE D.DName = 'Research' AND D.Dnumber = E.Dno;
```

Important Notes

- The **same attribute name** may be used in **different** relations — but **not** unambiguously in a multitable query **without qualification**.
- Qualification syntax: **<relation_name>.<attribute_name>** (e.g., EMPLOYEE.Name).
- **Fully qualifying** attribute names is good practice **even when not strictly required**, for **clarity**.
- **Aliases (tuple variables)** are declared either via **AS** or directly after the relation name in the FROM clause.
- Aliases are **essential** when a query must reference the **same relation more than once** (self-joins) — as in the supervisor-supervisee example (Q8).
- Aliases can also **rename attributes**, not just relations, by listing new attribute names after the alias.
- Using aliases for **every** table in a query — even when not strictly necessary — is **recommended practice** for readability.
- The self-join supervisor query is described as a **one-level recursive query**; general (unbounded-level) recursive queries were **not supported in early SQL**, but were added in **SQL:1999**.

SQL's mechanisms for attribute qualification and aliasing (tuple variables) resolve the practical ambiguity that arises when multiple relations share attribute names, or when a single relation must be referenced more than once within the same query — most notably in self-join queries such as retrieving each employee's supervisor from within the EMPLOYEE relation itself. Beyond resolving genuine ambiguity, fully qualifying attribute names and consistently using table aliases are considered good SQL practice

throughout, since they make multi-table queries substantially easier to read and verify — a habit well worth adopting even in queries where no ambiguity technically exists.

Missing WHERE Clause

A **missing WHERE clause** indicates that there is **no condition** on tuple selection; hence, **all tuples** of the relation(s) specified in the FROM clause **qualify** and are selected for the query result.

Important Note — Multiple Tables Without WHERE: If **more than one relation** is specified in the FROM clause, and there is **no WHERE clause**, then the **CROSS PRODUCT** — **all possible tuple combinations** — of these relations is selected.

Textbook Examples — Queries 9 and 10

Query 9: Select all EMPLOYEE Ssns.

```
Q9: SELECT Ssn
    FROM EMPLOYEE;
```

Query 10: Select all combinations of EMPLOYEE Ssn and DEPARTMENT Dname.

```
Q10: SELECT Ssn, Dname
     FROM EMPLOYEE, DEPARTMENT;
```

Explanation: Q9 simply retrieves the Ssn of **every** employee in the database, since there is no WHERE clause to filter tuples. Q10, however, involves **two** relations with **no WHERE clause**, so it selects **all combinations** of an EMPLOYEE Ssn and a DEPARTMENT Dname — **regardless of whether that particular employee actually works for that particular department or not**.

Important Warning: It is **extremely important** to specify **every** selection and join condition in the WHERE clause — if **any** such condition is **overlooked**, **incorrect and very large relations** may result. Note that Q10 closely resembles a **CROSS PRODUCT** operation followed by a **PROJECT** operation in relational algebra. If **all** attributes of EMPLOYEE and DEPARTMENT were specified in Q10 (rather than just Ssn and Dname), the result would be the **actual CROSS PRODUCT** (except for duplicate elimination, if any is applied).

The Asterisk (*)

To retrieve **all attribute values** of the selected tuples, it is **not necessary** to explicitly list every attribute name in SQL — instead, we can simply specify an **asterisk (*)**, which stands for **all the attributes**. The asterisk can also be **prefixed with a relation name or alias** — for example, **EMPLOYEE.*** refers specifically to all attributes of the EMPLOYEE table.

Textbook Examples:

Q1C: SELECT *

FROM EMPLOYEE

WHERE Dno = 5;

(Retrieves all attribute values of any EMPLOYEE who works in department number 5.)

Q1D: SELECT *

FROM EMPLOYEE, DEPARTMENT

WHERE Dname = 'Research' AND Dno = Dnumber;

(Retrieves all attributes of an EMPLOYEE, along with all attributes of the DEPARTMENT they work in, for every employee of the 'Research' department.)

Q10A: SELECT *

FROM EMPLOYEE, DEPARTMENT;

(Specifies the actual CROSS PRODUCT of the EMPLOYEE and DEPARTMENT relations, since all attributes of both tables are selected with no WHERE clause.)

4.3.3 Tables as Sets in SQL

SQL's Multiset Nature — Reasons for Not Auto-Eliminating Duplicates

As mentioned earlier, SQL usually treats a table **not as a set but as a multiset**; duplicate tuples can appear more than once, both in a stored table and in the **result of a query**. SQL does **not automatically eliminate duplicate tuples** in query results, for the following reasons:

- **Duplicate elimination is an expensive operation** — one common way to implement it is to first **sort** the tuples and then eliminate duplicates, which adds processing overhead.
- **The user may actually want to see duplicate tuples** in the result of a query, for various analytical or reporting purposes.
- When an **aggregate function** is applied to tuples, in **most cases** we do **not** want duplicates eliminated, since doing so would distort the aggregate result (e.g., a SUM or COUNT that silently drops repeated values would be incorrect for many use cases).

Important Note — When a Table IS Guaranteed to Be a Set: An SQL table **with a declared key** is restricted to behaving as a **true set**, since the key value is guaranteed to be **distinct in each tuple**. (In general, however, an SQL table is **not required** to have a key, although in most practical cases, one will typically be declared.)

The DISTINCT Keyword

If we **do** want to eliminate duplicate tuples from the result of a query, we use the keyword **DISTINCT** in the SELECT clause, meaning that only **distinct** tuples should remain in the result.

General Rule:

- A query with **SELECT DISTINCT → eliminates duplicates**.
- A query with **SELECT ALL** → does **NOT** eliminate duplicates.
- Specifying **SELECT** with **neither ALL nor DISTINCT** (as in most of the earlier examples) is **equivalent to SELECT ALL** — i.e., duplicates are retained by default.

Textbook Example — Query 11:

Q11: SELECT ALL Salary

FROM EMPLOYEE;

(Retrieves the salary of every employee — if several employees share the same salary, that value appears multiple times in the result.)

Q11A: SELECT DISTINCT Salary

```
FROM EMPLOYEE;
```

(Retrieves only distinct salary values — each unique salary appears exactly once in the result, regardless of how many employees earn it.)

SQL Set Operations — UNION, EXCEPT, INTERSECT

SQL has directly incorporated some of the **set operations from mathematical set theory**, which also form part of relational algebra: **UNION**, **set difference (EXCEPT)**, and **set intersection (INTERSECT)**.

Important Note: In some database systems, the keyword **MINUS** is used instead of **EXCEPT** for the set difference operation.

Key Properties: The relations resulting from these set operations are, by definition, **sets** of tuples — meaning **duplicate tuples are eliminated** from the result automatically. These set operations apply **only to type-compatible (union-compatible) relations** — the two relations involved must have the **same attributes**, appearing in the **same order**, in both relations.

Textbook Example — Query 4 Using UNION:

Query: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
Q4A: ( SELECT DISTINCT Pnumber
     FROM PROJECT, DEPARTMENT, EMPLOYEE
     WHERE Dnum = Dnumber AND Mgr_ssn = Ssn
          AND Lname = 'Smith' )
   UNION
   ( SELECT DISTINCT Pnumber
     FROM PROJECT, WORKS_ON, EMPLOYEE
     WHERE Pnumber = Pno AND Essn = Ssn
          AND Lname = 'Smith' );
```

Explanation: The **first** SELECT query retrieves the projects that involve a 'Smith' as **manager of the controlling department**; the **second** retrieves the projects that involve a 'Smith' as a **worker** on the project directly. Applying the **UNION** operation to these two SELECT queries combines both sets, giving the desired final result — with duplicates automatically removed. **Note:** if several employees share the last name 'Smith', the project numbers involving **any** of them will be retrieved.

Multiset Variants — UNION ALL, EXCEPT ALL, INTERSECT ALL

SQL also provides corresponding **multiset operations**, formed by appending the keyword **ALL**: **UNION ALL, EXCEPT ALL, INTERSECT ALL**. Their results are treated as **multisets** — meaning **duplicates are NOT eliminated**. Under these ALL-variants, **each tuple — whether or not it is a duplicate — is considered a distinct, individual tuple** when the operation is applied, rather than being merged or filtered based on equality with other tuples.

- **Missing WHERE clause** → all tuples of a single relation qualify; for **multiple** relations with no WHERE, the result is the **CROSS PRODUCT** of all listed relations.
- **Asterisk (*)** retrieves **all attributes**; can be prefixed with a relation/alias name (e.g., EMPLOYEE.*).
- SQL tables are, by default, **multisets (bags)**, not sets — duplicates are **not** automatically eliminated.
- Reasons duplicates are kept by default: **cost of elimination**, **user may want duplicates**, and **aggregate functions usually need duplicates preserved**.
- A table **with a declared key** is guaranteed to behave as a true **set** (no duplicate rows possible).
- **DISTINCT** in SELECT eliminates duplicates; **ALL** (or nothing specified) retains them.
- **UNION, EXCEPT (or MINUS), INTERSECT** — SQL's set operations — always **eliminate duplicates** and require **union-compatible (type-compatible)** relations.

- **UNION ALL, EXCEPT ALL, INTERSECT ALL** — the **multiset** counterparts — do **NOT** eliminate duplicates.

This part highlights two important practical dimensions of SQL retrieval: first, that omitting selection or join conditions in the WHERE clause — or omitting the WHERE clause entirely — can silently produce a CROSS PRODUCT of unrelated tuple combinations, making careful condition-writing essential; and second, that SQL fundamentally treats tables as multisets rather than true mathematical sets, retaining duplicate tuples by default unless DISTINCT is explicitly specified or a key constraint guarantees uniqueness. SQL's set operations — UNION, EXCEPT, and INTERSECT — bridge back to the formal relational algebra by always eliminating duplicates and requiring union-compatible relations, while their ALL-suffixed multiset counterparts preserve every individual tuple, duplicates included, giving the SQL programmer precise control over whether or not duplicate elimination is desired in any given query.

4.3.4 SUBSTRING PATTERN MATCHING

Introduction

In many real-world queries, a user may not know the **exact value** stored in a database, but only a **partial value or pattern**. For example, a user may want to retrieve all employees whose address contains "Houston" without knowing the complete address string. SQL provides a special comparison operator called **LIKE** for this purpose, along with two **wildcard characters** used to construct string patterns.

Since the = (equality) operator can only compare **exact string values**, SQL introduces pattern matching to allow **partial/fuzzy string comparisons**, which are essential in real applications like search engines, filtering by name, address matching, etc.

The LIKE Operator

The general syntax is:

<attribute name> LIKE <pattern>

or, for exclusion:

<attribute name> NOT LIKE <pattern>

Wildcard Characters

SQL defines **two reserved characters** for constructing the pattern string:

1. **% (percent sign)** — Replaces an **arbitrary number of characters** (zero or more characters), including no characters at all.
2. **_ (underscore)** — Replaces **exactly one single character**.

These characters can be used in any combination and at any position within the pattern string.

Rules for Constructing Patterns

Symbol	Meaning
%	zero, one, or many characters
_	exactly one character
'ABC%'	starts with ABC
'%ABC'	ends with ABC
'%ABC%'	contains ABC anywhere
'A_C'	A, then any one character, then C (3 characters total)

Example Table – EMPLOYEE

Fname	Lname	Address
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX

Illustrative Queries

Query 1: Retrieve all employees whose address is in Houston, Texas.

sql

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Address LIKE '%Houston, TX%';
```

Here % before and after ensures "Houston, TX" can occur anywhere within the Address string, with any number of characters before and after it.

Query 2: Retrieve employees born during the 1950s whose Ssn (Social Security Number, a 9-digit string) has '50' in positions 7 and 8 of the string.

sql

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Ssn LIKE '_______50_____';
```

Here, each underscore stands for exactly one unknown character, precisely fixing the position of '50' within the string.

Query 3: Employees whose last name starts with "S":

sql

```
SELECT * FROM EMPLOYEE
WHERE Lname LIKE 'S%';
```

Query 4: Employees whose last name has exactly 5 characters and starts with "S":

sql

```
SELECT * FROM EMPLOYEE
WHERE Lname LIKE 'S____';
```

(1 fixed character + 4 underscores = 5 total characters)

Important Points (Exam Notes)

- LIKE is used specifically for **string/character type attributes**.
- Pattern matching is essential when the **exact value is unknown**, but a partial pattern is known.
- % and _ are **reserved wildcard characters**; if the actual data contains these characters literally, an **ESCAPE clause** is used:

sql

```
SELECT * FROM PRODUCT
```

```
WHERE Description LIKE '%50\%%' ESCAPE '\';
```

This searches for the literal string "50%" using \ as the escape character.

- LIKE comparisons are generally **case-sensitive**, depending on the DBMS and collation settings.
- NOT LIKE is used to retrieve rows that **do not match** a given pattern.

4.3.5 ARITHMETIC OPERATORS

Introduction

SQL allows the use of the standard **arithmetic operators** +, –, *, and / for addition, subtraction, multiplication, and division respectively. These operators can be applied to numeric attributes in the SELECT clause to **compute and display derived values** that are not directly stored in the database but calculated at query time.

This is useful because databases often store *raw* data (e.g., salary, hours), and users frequently need *computed/derived* results (e.g., salary after increment, total pay, etc.) without altering the stored data.

General Syntax

sql

```
SELECT column1, column2, (arithmetic_expression) AS alias_name
FROM table_name
WHERE condition;
```

Example Table – EMPLOYEE

Fname	Lname	Salary	Dno
John	Smith	30000	5
Franklin	Wong	40000	5
Alicia	Zelaya	25000	4

Illustrative Queries

Query 1: Show the salary of every employee, along with the salary increased by 10%, for employees working in department 5.

sql

```
SELECT Fname, Lname, Salary, 1.1 * Salary AS Increased_Sal
FROM EMPLOYEE
WHERE Dno = 5;
```

Result concept:

Fname	Lname	Salary	Increased_Sal
John	Smith	30000	33000
Franklin	Wong	40000	44000

Query 2: Compute annual salary for all employees.

sql

```
SELECT Fname, Lname, Salary * 12 AS Annual_Salary
FROM EMPLOYEE;
```

Query 3: Arithmetic expressions can also be used within the WHERE clause for filtering based on computed values.

sql

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE (Salary * 12) > 500000;
```

Query 4: Compute net salary after deduction:

sql

```
SELECT Fname, Lname, (Salary + Bonus - Deduction) AS Net_Salary
FROM EMPLOYEE;
```

Important Points (Exam Notes)

- Arithmetic operators can be applied only to **numeric-type attributes** (integer, float, decimal, etc.).
- Standard **operator precedence** applies: * and / are evaluated before + and –; parentheses () can be used to override precedence.
- The result of an arithmetic expression is a **derived/computed attribute**, not a stored column; it is typically given a meaningful name using the AS clause (**aliasing**).
- Arithmetic operators can appear in:
  - The SELECT clause (to display computed values)
  - The WHERE clause (to filter based on computed conditions)
- These operations **do not modify** the underlying stored data — they only affect the query's output/display.

4.3.6 ORDERING OF QUERY RESULTS (ORDER BY)

Introduction

The basic SELECT query in SQL does **not guarantee any specific order** for the tuples in the result — the order in which rows are returned is essentially arbitrary and depends on internal storage/execution details of the DBMS. To **display or arrange** query results in a **particular sequence**, SQL provides the ORDER BY clause.

General Syntax

sql

SELECT column_list

FROM table_name

[WHERE condition]

ORDER BY column1 [ASC | DESC], column2 [ASC | DESC], ...;

- ASC → Ascending order (**default**, if not specified)
- DESC → Descending order

Example Table – EMPLOYEE

Dno	Lname	Fname	Salary
5	Smith	John	30000
5	Wong	Franklin	40000
4	Zelaya	Alicia	25000
4	Wallace	Jennifer	43000

Illustrative Queries

Query 1: Retrieve a list of employees and the projects they work on, ordered by department and, within each department, ordered alphabetically by last name and first name.

sql

```
SELECT Dno, Lname, Fname
FROM EMPLOYEE
ORDER BY Dno, Lname, Fname;
```

Query 2: List employees in decreasing order of salary.

sql

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
ORDER BY Salary DESC;
```

Query 3: Order by department ascending, and within each department, order salary descending (major key + minor key sorting):

sql

```
SELECT Dno, Lname, Salary
FROM EMPLOYEE
ORDER BY Dno ASC, Salary DESC;
```

Result concept:

Dno	Lname	Salary
4	Wallace	43000
4	Zelaya	25000
5	Wong	40000
5	Smith	30000

- **Sort keys**: The first column listed in ORDER BY is called the **major sort key**; subsequent columns are used to break ties and are called **minor sort keys**.
- If multiple attributes are specified, sorting is performed **hierarchically** — first sorted by the first attribute, then, for rows with equal values in that attribute, sorted by the next attribute, and so on.
- ORDER BY is applied **last**, after all other clauses (SELECT, FROM, WHERE, GROUP BY, HAVING) have been processed — because sorting only makes sense on the final filtered/aggregated result set.
- You may specify column **position numbers** instead of column names:

sql

```
SELECT Fname, Lname, Salary FROM EMPLOYEE ORDER BY 3 DESC;
```

(orders by the 3rd column in the SELECT list, i.e., Salary) — though using explicit column names is considered better practice for readability and maintainability.

- Default sort order is **ascending (ASC)** if not explicitly specified.

Combined Example (Integrating All Three Concepts)

sql

```
SELECT Fname, Lname, Salary, (Salary * 12) AS Annual_Salary
FROM EMPLOYEE
WHERE Lname LIKE '%son'
ORDER BY Annual_Salary DESC;
```

Explanation: This query retrieves employees whose last name ends with "son" (substring pattern matching), computes their annual salary from monthly salary

(arithmetic operator), and arranges the final result in descending order of annual salary (ordering clause).

INSERT, DELETE, and UPDATE Statements in SQL, and Additional Features of SQL

Besides retrieving data (SELECT), SQL provides three commands to **modify the contents of the database**:

1. **INSERT** — adds new tuples (rows) to a relation
2. **DELETE** — removes existing tuples from a relation
3. **UPDATE** — modifies attribute values of existing tuples

These three commands, together with SELECT, form the core of SQL's **Data Manipulation Language (DML)**.

4.4 THE INSERT COMMAND

Basic Purpose

The INSERT command is used to add a **single tuple** (or multiple tuples) to a relation. In its simplest form, we specify the relation name and a list of values for the new tuple.

Form 1: Insert Without Specifying Attribute Names

```
INSERT INTO EMPLOYEE
VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30',
    '98 Oak Forest, Katy, TX', 'M', 37000, '653298653', 4);
```

Key rule: The values **must be listed in the exact same order** in which the corresponding attributes were defined in the CREATE TABLE statement. If even one value is misplaced, it will be inserted into the wrong column.

Form 2: Insert With Explicit Attribute Names

SQL allows specifying **attribute names** explicitly along with their corresponding values. This is useful when:

- A relation has many attributes, but only a few are known/available for the new tuple.

- We want to insert values in an order **different** from the table definition.

INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)

VALUES ('Richard', 'Marini', 4, '653298653');

Rules:

- Attributes **not specified** in the command are automatically set to their **DEFAULT value** (if defined) or to **NULL**.
- All attributes with a **NOT NULL** constraint and **no default value** *must* be included in the value list — they cannot be omitted.
- Attributes that allow **NULL** or have a **DEFAULT** value can be safely left out.
- Values must be listed in the **same order** as the attribute names given in the INSERT INTO clause itself (not necessarily the table's original order).

Form 3: Inserting Multiple Tuples in One Command

Multiple tuples can be inserted in a **single INSERT statement**, separated by commas, with each tuple's values enclosed in parentheses:

INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)

VALUES ('Richard', 'Marini', 4, '653298653'),

('Alicia', 'Zelaya', 4, '999887777'),

('Ahmad', 'Jabbar', 4, '987654321');

Enforcement of Integrity Constraints

A DBMS that fully implements SQL enforces **all integrity constraints** defined in the DDL during an INSERT operation.

Example — Referential Integrity Violation:

INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno)

VALUES ('Robert', 'Hatcher', '980760540', 2);

This is **rejected** if no DEPARTMENT tuple exists with Dnumber = 2, because it violates the **referential integrity (foreign key)** constraint.

Example — NOT NULL / Primary Key Violation:

```
INSERT INTO EMPLOYEE (Fname, Lname, Dno)
VALUES ('Robert', 'Hatcher', 5);
```

This is **rejected** because Ssn is the **primary key** and cannot be NULL, but no Ssn value was provided.

Inserting Data Using a Query (INSERT ... SELECT)

A powerful variation of INSERT allows loading a table with the **result of a query**, rather than manually specifying values. This is often used to create and populate temporary/summary tables.

Step 1 — Create the table:

```
CREATE TABLE WORKS_ON_INFO
( Emp_name        VARCHAR(15),
  Proj_name       VARCHAR(15),
  Hours_per_week  DECIMAL(3,1) );
```

Step 2 — Populate it using a SELECT query:

```
INSERT INTO WORKS_ON_INFO (Emp_name, Proj_name, Hours_per_week)
SELECT E.Lname, P.Pname, W.Hours
FROM PROJECT P, WORKS_ON W, EMPLOYEE E
WHERE P.Pnumber = W.Pno AND W.Essn = E.Ssn;
```

Explanation: The table WORKS_ON_INFO is created empty, then loaded with joined data (employee last name, project name, hours per week) retrieved from PROJECT, WORKS_ON, and EMPLOYEE.

Important note: This new table is a **static snapshot** — it is **not automatically kept up to date**. If the source tables (PROJECT, WORKS_ON, EMPLOYEE) are later updated, WORKS_ON_INFO will become **outdated/stale**. To always have current derived data, a **VIEW** should be created instead (covered in Chapter 7).

Once no longer needed, such a table can be removed with:

```
DROP TABLE WORKS_ON_INFO;
```

Bulk Loading Data

- Most DBMSs provide **bulk loading tools/utilities** that let users load large volumes of formatted data from an external file into a table **without writing individual INSERT statements** for every row.
- Alternatively, a program can be written (using embedded SQL or a database API) to read each record from a file, format it, and insert it using **looping constructs** of a programming language.

The LIKE Clause — Creating a Table Structurally Similar to Another

SQL allows creating a **new table with the same structure** as an existing table, and simultaneously loading it with a subset of that table's data, using the LIKE clause.

Example: Create table D5EMPS with the same structure as EMPLOYEE, loaded only with employees from department 5:

```
CREATE TABLE D5EMPS LIKE EMPLOYEE
(SELECT E.*
 FROM EMPLOYEE AS E
 WHERE E.Dno = 5) WITH DATA;
```

- LIKE EMPLOYEE copies the **column structure** (not the constraints, necessarily) of EMPLOYEE.
- WITH DATA specifies that the table should be **populated immediately** with the result of the subquery.
- In some SQL implementations, the WITH DATA clause may be **optional or omitted**.

4.4.1 THE DELETE COMMAND

Purpose

The DELETE command removes **one or more tuples** from a **single relation**. It uses a WHERE clause (similar to a SELECT query) to specify **which** tuples should be deleted.

General Syntax

```
DELETE FROM table_name
```

[WHERE condition];

Key Characteristics

- Tuples are deleted from **only one table at a time** in a single DELETE command.
- However, deletion can **propagate** to related tuples in **other tables** if **referential triggered actions** (like ON DELETE CASCADE) are specified as part of referential integrity constraints in the DDL.
- Depending on how many tuples satisfy the WHERE condition, a DELETE command may remove **zero, one, several, or all** tuples.
- **A missing WHERE clause** means **all tuples** in the relation are deleted — however, the **table itself still exists** as an **empty table** in the database. To remove the table definition entirely, the DROP TABLE command must be used separately.

Illustrative Examples

Applied independently to a sample database state:

```
-- U4A: Deletes 0 tuples (if no employee has this last name)
DELETE FROM EMPLOYEE
WHERE Lname = 'Brown';
-- U4B: Deletes exactly 1 tuple (Ssn is a primary key → unique match)
DELETE FROM EMPLOYEE
WHERE Ssn = '123456789';
-- U4C: Deletes multiple tuples (all employees in department 5)
DELETE FROM EMPLOYEE
WHERE Dno = 5;
-- U4D: Deletes ALL tuples in the EMPLOYEE table (table remains, but empty)
DELETE FROM EMPLOYEE;
```

Exam Tip: Distinguish clearly —

- DELETE FROM table; → removes all **rows**, table structure remains.

- DROP TABLE table; → removes the **entire table** (structure + data) from the schema.

4.4.2 THE UPDATE COMMAND

Purpose

The UPDATE command modifies the **attribute values** of one or more **selected tuples** in a relation.

General Syntax

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

Key Characteristics

- Like DELETE, the WHERE clause selects **which tuples** are to be modified, and the command applies to a **single relation** only.
- The **SET clause** specifies which attributes are to be changed and their new values.
- Updating a **primary key** value may **propagate** to related **foreign key** values in other tables, if a referential triggered action (e.g., ON UPDATE CASCADE) is defined in the DDL.
- Multiple attributes can be updated simultaneously within one SET clause.
- Each UPDATE statement refers to **only one table**; to modify multiple tables, **separate UPDATE commands** must be issued for each.

Example 1 — Updating a Single Tuple

Change the location and controlling department number of project number 10:

UPDATE PROJECT

SET Plocation = 'Bellaire', Dnum = 5

WHERE Pnumber = 10;

Example 2 — Updating Multiple Tuples with a Computed Value

Give all employees in department 5 (Research) a 10% salary raise:

UPDATE EMPLOYEE

SET Salary = Salary * 1.1

WHERE Dno = 5;

Important conceptual point: In this example, the **new** Salary value **depends on the old** Salary value. SQL handles this correctly:

- The Salary on the **right-hand side** of = refers to the **old (original) value** before the update.
- The Salary on the **left-hand side** refers to the **new value** being assigned.

This allows computing updated values based on existing stored values, without needing to know them beforehand.

Additional Notes

- It is also valid to set an attribute's new value to **NULL** or to its **DEFAULT** value:
- UPDATE EMPLOYEESET Super_ssn = NULLWHERE Ssn = '123456789';
- As with DELETE, **omitting the WHERE clause** in UPDATE causes the SET operation to be applied to **every tuple** in the table.

INSERT, DELETE, UPDATE

Command	**Purpose**	**Affects**
INSERT INTO ... VALUES(...)	Add new tuple(s)	One relation
INSERT INTO ... SELECT ...	Add tuples from query results	One relation (source: query)
DELETE FROM ... WHERE ...	Remove tuple(s)	One relation

Command	Purpose	Affects
UPDATE ... SET ... WHERE ...	Modify attribute value(s) of tuple(s)	One relation

Common Features:

- All three commands can use a WHERE clause (except basic single-tuple INSERT) to target specific rows.
- All are subject to **integrity constraint checking** (NOT NULL, PRIMARY KEY, FOREIGN KEY/referential integrity).
- Referential integrity violations, foreign key mismatches, or NOT NULL violations cause the DBMS to **reject** the operation.
- Referential **triggered actions** (CASCADE, SET NULL, etc.) can cause DELETE/UPDATE operations to **propagate** to related tables.

4.5 ADDITIONAL FEATURES OF SQL (Overview)

This section briefly previews advanced SQL topics covered in later chapters of the textbook:

Feature	Description	Covered In
Complex retrieval queries	Nested queries, aggregate functions, grouping, joined tables, outer joins, CASE statements, recursive queries	Chapter 7
Views, Triggers, Assertions	Virtual tables, active rules, and constraint enforcement mechanisms	Chapter 7
Schema modification commands	ALTER, DROP, etc.	Chapter 7

Feature	Description	Covered In
Embedded/Dynamic SQL	Writing programs in host languages (C, Java, etc.) that include SQL statements	Chapter 10
SQL/CLI, ODBC, SQL/PSM	Call-level interfaces and stored procedures for database connectivity	Chapter 10
JDBC, SQLJ	Accessing SQL databases via the Java programming language	Chapter 10
Storage Definition Language (SDL)	Commands for physical design — file structures, indexes (e.g., CREATE INDEX)	Chapter 17
Transaction control commands	COMMIT, ROLLBACK — used for concurrency control and recovery	Chapter 20
GRANT / REVOKE	Language constructs for granting/revoking user privileges (SELECT, INSERT, DELETE, UPDATE)	Chapter 20
Triggers (Active Databases)	Automatic actions triggered by database events (e.g., updates)	Chapter 26 (Section 26.1)
Object-Relational Features	UDTs (user-defined types), complex-structured attributes, object identifiers	Chapter 12

Feature	Description	Covered In
XML and OLAP/Data Warehousing	SQL's interaction with newer technologies	Chapters 13, 29

Command/Feature	Keyword	Notes
Insert single tuple	INSERT INTO ... VALUES	Order must match table definition
Insert with named attributes	INSERT INTO table(col1,col2) VALUES	Missing attrs → NULL/DEFAULT
Insert multiple tuples	VALUES (...), (...), (...)	Comma-separated tuples
Insert via query	INSERT INTO ... SELECT ...	Data may become stale (use VIEW to stay current)
Table with same structure	CREATE TABLE ... LIKE ... WITH DATA	Copies structure + optional data
Delete tuples	DELETE FROM ... WHERE	No WHERE → deletes all rows (table stays)
Remove table entirely	DROP TABLE	Removes structure + data
Update tuples	UPDATE ... SET ... WHERE	RHS = old value, LHS = new value
Set to NULL/DEFAULT	SET col = NULL	Valid for UPDATE

4.6 Informal Design Guidelines for Relation Schemas

Introduction

Before studying the **formal theory** of relational database design (functional dependencies, normal forms, BCNF), it is useful to first understand **four informal**

guidelines that can be used as **measures to judge the quality** of a relation schema design. These guidelines don't require formal mathematical tools — they can be applied by simply examining the relation schema and its typical data.

The **four informal guidelines** are:

1. Making sure the **semantics of attributes** is clear in the schema
2. **Reducing redundant information** in tuples
3. **Reducing NULL values** in tuples
4. **Disallowing the possibility of generating spurious tuples**

Important note: These four measures are **not always independent** of one another — improving one often improves the others, and violating one guideline can lead to violations of the others as well.

4.6.1 Imparting Clear Semantics to Attributes in Relations

Concept of Semantics

Whenever attributes are grouped together to form a relation schema, we implicitly assume that the attributes belonging to that relation have **some clear real-world meaning** and a **proper interpretation** associated with them.

Semantics of a relation = the meaning that results from interpreting the attribute values in a tuple of that relation.

A relation can be thought of as representing a **set of facts**. If conceptual design (ER modeling) is done carefully, and the ER-to-relational mapping is applied systematically, the resulting relational schema will naturally have **clear semantics**.

Illustration — The COMPANY Database

Consider a simplified COMPANY schema with the relations:

- **EMPLOYEE** (Ename, Ssn, Bdate, Address, Dnumber)
- **DEPARTMENT** (Dname, Dnumber, Dmgr_ssn)
- **PROJECT** (Pname, Pnumber, Plocation, Dnum)
- **DEPT_LOCATIONS** (Dnumber, Dlocation)
- **WORKS_ON** (Ssn, Pnumber, Hours)

Meaning of each relation:

- Each **EMPLOYEE** tuple represents one employee, with Dnumber as a **foreign key** linking to the department the employee works for.
- Each **DEPARTMENT** tuple represents one department entity, with Dmgr_ssn (foreign key) identifying the employee who manages it.
- Each **PROJECT** tuple represents one project entity, with Dnum (foreign key) identifying the controlling department.
- **DEPT_LOCATIONS** represents a **multivalued attribute** of DEPARTMENT (a department can have multiple locations).
- **WORKS_ON** represents an **M:N relationship** between EMPLOYEE and PROJECT (an employee can work on many projects, and a project can have many employees), along with the Hours attribute of that relationship.

Since each of these relations corresponds to **exactly one entity type or one relationship type**, their meaning is **easy to explain** — this is a hallmark of good design.

Guideline 1

Guideline 1: Design a relation schema so that it is **easy to explain its meaning**. Do **not** combine attributes from **multiple entity types and relationship types** into a single relation. If a relation schema corresponds to **one entity type** or **one relationship type**, its meaning is straightforward. If it corresponds to a **mixture** of multiple entities/relationships, semantic ambiguity results, and the schema becomes hard to explain.

Example — Violating Guideline 1

Consider two "combined" relations:

EMP_DEPT (Ename, Ssn, Bdate, Address, **Dnumber, Dname, Dmgr_ssn**)

- Each tuple represents a single employee, but it also carries the **department name** and **manager's Ssn** — attributes that logically belong to the DEPARTMENT entity, not the employee.

EMP_PROJ (Ssn, **Pnumber, Hours**, Ename, Pname, Plocation)

- Each tuple relates an employee to a project (from WORKS_ON) but also includes the **employee's name**, and the **project's name and location** — attributes that belong to EMPLOYEE and PROJECT respectively.

Problem: Although these relations are **not logically incorrect** (they contain valid, consistent data), they **violate Guideline 1** because:

- EMP_DEPT mixes attributes of **EMPLOYEE** and **DEPARTMENT**.
- EMP_PROJ mixes attributes of **EMPLOYEE**, **PROJECT**, and the **WORKS_ON** relationship.

Such relations may be acceptable as **views** (virtual, derived relations) but cause serious problems if used as **base relations** (physically stored tables) — as explained in the next section.

4.6.2 Redundant Information in Tuples and Update Anomalies

The Problem of Redundancy

One important goal of schema design is to **minimize storage space** used by base relations. The way attributes are grouped into relations has a **direct effect** on redundancy and storage.

Example: Compare:

- The **separate** EMPLOYEE and DEPARTMENT relations, vs.
- A **combined** EMP_DEPT relation formed by applying **NATURAL JOIN** on EMPLOYEE and DEPARTMENT.

In EMP_DEPT, the department-related values (**Dnumber, Dname, Dmgr_ssn**) are **repeated for every employee** who belongs to that department. In contrast, in the separate design, each department's data appears **only once** in DEPARTMENT; only the Dnumber (a small foreign key value) is repeated in EMPLOYEE.

The same redundancy issue applies to EMP_PROJ, which merges WORKS_ON with extra attributes from EMPLOYEE and PROJECT.

This unnecessary duplication of data leads to a set of problems collectively called **update anomalies**. These were first identified by **Codd (1972)** to justify the need for **normalization**. Update anomalies are classified into **three types**:

(a) Insertion Anomalies

Two distinct sub-problems, illustrated using EMP_DEPT:

Problem 1 — Difficulty inserting a new employee: To insert a new employee tuple, we must **correctly repeat** all attribute values of that employee's department (Dname, Dmgr_ssn) so that they remain **consistent** with existing tuples for the same department. This creates a **consistency risk** — if the department data is entered incorrectly or inconsistently for different employee tuples, the database becomes contradictory.

(In the normalized design, this problem doesn't arise — the employee tuple only stores the Dnumber, and department details are recorded exactly once in DEPARTMENT.)

Problem 2 — Difficulty inserting a new department with no employees yet: To insert a department that currently has **no employees**, the only way in EMP_DEPT is to insert a tuple with **NULL** values for all employee attributes. But this **violates entity integrity**, since the primary key Ssn of EMP_DEPT **cannot be NULL**. Also, once the first employee is assigned to that department, this placeholder tuple becomes unnecessary and must be removed.

(This problem does not occur in the normalized design — a department can be inserted into DEPARTMENT independently, whether or not it has employees yet.)

(b) Deletion Anomalies

Related to insertion Problem 2. If we **delete** the EMP_DEPT tuple of the **last remaining employee** of a particular department, **all information about that department is lost inadvertently** — since department data existed only as part of that employee's tuple.

(This does not happen in the normalized design, since DEPARTMENT tuples are stored independently of EMPLOYEE tuples.)

(c) Modification Anomalies

If an attribute value of a department needs to change (e.g., changing the **manager of department 5**), we must update **every single employee tuple** belonging to that department. If even **one tuple is missed**, the database becomes **inconsistent** — showing two different managers for the same department in different tuples.

(This is considered a relatively less serious anomaly since it can typically be handled by a single SQL UPDATE statement with a WHERE clause affecting multiple rows — but it still represents unnecessary, avoidable redundancy.)

Guideline 2

Guideline 2: Design base relation schemas so that **no insertion, deletion, or modification anomalies** are present. If anomalies **cannot be avoided**, they should be **clearly documented**, and the application programs that update the database must be written to **handle them correctly** (e.g., via triggers or stored procedures).

Important Practical Note

Guideline 2 is essentially a **restatement/consequence** of Guideline 1. However, in **real-world systems**, these guidelines may sometimes need to be **deliberately violated for performance reasons** — e.g., using EMP_DEPT as a **materialized view** to speed up frequently-run queries/reports. In such cases:

- The anomalies must be **explicitly accounted for**.
- Automatic mechanisms (**triggers, stored procedures**) should be used to keep the materialized/denormalized data **consistent** with the base tables.
- General best practice: keep **base relations anomaly-free** (normalized), and create **views** that perform the necessary joins for frequently needed queries.

4.6.3 NULL Values in Tuples

The Problem

If a schema design groups **too many attributes together into one "fat" relation**, and many of those attributes **do not apply to all tuples**, the result is a large number of **NULL values** scattered throughout the table.

Consequences of Excessive NULLs

1. **Wasted storage space** at the physical/file level.
2. **Difficulty understanding attribute meaning** and specifying **JOIN operations** correctly at the logical level (inner vs. outer joins behave differently with NULLs, which can confuse users).
3. **Problems with aggregate functions** like COUNT and SUM — NULLs must be specially accounted for and can produce unexpected results.
4. **Unpredictable results in SELECT and JOIN operations**, since comparisons involving NULL follow **three-valued logic** (TRUE, FALSE, UNKNOWN) rather than simple TRUE/FALSE.

Multiple Interpretations of NULL

A NULL value can mean different things, and using the **same representation** for all of them causes **ambiguity**:

Interpretation	Example
Attribute does not apply to this tuple	Visa_status doesn't apply to U.S. students
Value is unknown	Date_of_birth unknown for an employee
Value exists but is not yet recorded	Home_Phone_Number exists but hasn't been entered

Guideline 3

Guideline 3: As far as possible, **avoid placing attributes in a base relation whose values may frequently be NULL**. If NULLs are unavoidable, ensure they occur only in **exceptional cases**, not for the **majority** of tuples.

Design Strategy

The two overriding criteria for deciding whether an attribute (prone to NULLs) should be: (a) kept in the main relation, or (b) placed in a **separate relation** (with the appropriate key)

...are:

- **Efficient use of storage space**
- **Avoiding problematic joins involving NULLs**

Example: If only **15% of employees** have an individual office, it is wasteful/inefficient to include an Office_number attribute directly in the EMPLOYEE relation (which would leave it NULL for 85% of rows). Instead, create a separate relation:

EMP_OFFICES (Essn, Office_number)

This relation would contain tuples **only for employees who actually have an office**, eliminating the NULLs entirely.

4.6.4 Generation of Spurious Tuples

Setting Up the Example

Consider decomposing the earlier EMP_PROJ relation into **two smaller relations**:

- **EMP_LOCS** (Ename, Plocation) — meaning: employee Ename works on **at least one** project located at Plocation.
- **EMP_PROJ1** (Ssn, Pnumber, Hours, Pname, Plocation) — meaning: employee Ssn works Hours per week on project Pname/Pnumber, located at Plocation.

These are obtained by applying **PROJECT (π)** operations on the original EMP_PROJ relation.

The Problem

If EMP_LOCS and EMP_PROJ1 are used as the **base relations** (instead of storing EMP_PROJ directly), a serious problem arises: **we cannot correctly recover the original EMP_PROJ information** by joining them back together.

When a **NATURAL JOIN** is performed on EMP_LOCS and EMP_PROJ1 (joining on the common attribute Plocation), the result produces **far more tuples** than existed in the original EMP_PROJ relation. These extra, incorrect tuples are called **spurious tuples**.

Why does this happen? The join is being performed on Plocation — but Plocation is **neither a primary key nor a foreign key** in either EMP_LOCS or EMP_PROJ1. It is simply a **shared descriptive attribute**, and multiple unrelated employees/projects can coincidentally share the same location value. Joining on such a "coincidentally matching" attribute produces **combinations of data that never actually existed together**, generating **invalid/spurious information**.

Guideline 4

Guideline 4: Design relation schemas so they can be joined using **equality conditions on attributes that form proper (primary key, foreign key) pairs**, guaranteeing that **no spurious tuples** are generated. **Avoid** relations containing matching attributes that are **not** in a (foreign key, primary key) relationship, since joining on such attributes may produce spurious tuples.

Formal Connection

This guideline is stated only **informally** here. It is formalized later (Chapter 15) using the concept of the **nonadditive join property** (also called the **lossless join property**) — a formal condition that guarantees a decomposition of a relation can be joined back together **without generating spurious tuples** and **without losing information**.

4.6.5 Summary and Discussion of Design Guidelines

The problems identified through Sections 14.1.1–14.1.4 — all detectable **without any formal analytical tools** — can be summarized as:

1. **Anomalies** (insertion, deletion, modification) that cause **redundant work** during updates and can cause **accidental loss of information** during deletions.
2. **Wasted storage space** due to excessive **NULLs**, plus difficulty performing **selections, aggregations, and joins**.
3. **Generation of invalid/spurious data** during joins on base relations that share attributes **not** representing a genuine (foreign key, primary key) relationship.

Looking Ahead

These four guidelines are **informal** — useful as a first check, but not mathematically rigorous. The rest of Chapter 14 (and Chapter 15) introduces the **formal theoretical framework** needed to evaluate relational schema quality precisely:

- **Functional Dependencies (FDs)** — the fundamental tool of analysis (Section 14.2)
- **Normal Forms**: 1NF, 2NF, 3NF, and **Boyce-Codd Normal Form (BCNF)** — accepted standards of relational design quality
- The general design **strategy**: start with a poorly designed ("unnormalized") relation and **decompose** it systematically to achieve higher normal forms, removing anomalies in the process.
- Chapter 15 covers **decomposition properties** in detail, along with algorithms for functional dependency analysis, measuring "goodness" of decomposition, and **bottom-up relational design** using FDs as the starting point.

Quick Revision Table — The Four Informal Guidelines

#	Guideline	Problem Addressed	Key Term
1	Ensure clear semantics — one relation = one entity/relationship type	Ambiguous meaning when mixing entities	Semantics
2	Avoid insertion, deletion, modification anomalies	Redundant/inconsistent updates, accidental data loss	Update Anomalies
3	Minimize NULLs in base relations	Wasted space, join/aggregate issues, ambiguous NULL meaning	NULL Values
4	Join only on proper (PK, FK) pairs	Invalid extra tuples from joining on non-key common attributes	Spurious Tuples

Update Anomalies — At a Glance

Anomaly Type	Cause	Example (EMP_DEPT)
Insertion	Can't add data without related data / can't add entity with no related entity	Can't add dept. w/o employee; must repeat dept info per employee
Deletion	Deleting one fact unintentionally deletes another	Deleting last employee of a dept. loses all dept info
Modification	Same fact stored redundantly must be updated everywhere	Changing dept manager requires updating every employee row of that dept

4.7 FUNCTIONAL DEPENDENCIES

4.7.1 Definition of Functional Dependency

Concept of the Universal Relation

Suppose a database schema has *n* attributes A_1, A_2, ..., A_n. For the purpose of formal theory, we imagine the **entire database** as being described by a **single universal relation schema**:

R = {A1, A2, ..., An}

Important clarification: This does **not** mean the database is actually **stored** as one giant table — the universal relation is just a **conceptual/theoretical device** used to develop the formal theory of data dependencies. (It becomes important later in Chapter 15's design algorithms.) This assumption implies every attribute in the database should have a **distinct name**.

Formal Definition

Definition: A functional dependency, denoted **X → Y**, between two sets of attributes X and Y (both subsets of R), specifies a **constraint** on the possible tuples that can form a relation state *r* of R. The constraint is: for **any two tuples** t_1 and t_2 in *r*, if **t_1[X] = t_2[X]**, then it **must also be true** that **t_1[Y] = t_2[Y]**.

Interpretation

- The values of the **Y component** of a tuple **depend on** or are **determined by** the values of the **X component**.
- Equivalently: the values of X **uniquely (functionally) determine** the values of Y.
- We say: **"there is a functional dependency from X to Y"**, or **"Y is functionally dependent on X"**.
- **Abbreviation:** FD or f.d.
- **X** = the **left-hand side (LHS)** of the FD
- **Y** = the **right-hand side (RHS)** of the FD

So, **X functionally determines Y** in schema R **if and only if** whenever two tuples of r(R) **agree on their X-value**, they **necessarily agree** on their Y-value too.

1. **Relationship with candidate keys:** If a constraint states that **no two tuples** can have the same X-value in any relation instance r(R) — i.e., **X is a candidate key** of R — then this implies **X → Y** for **any** subset of attributes Y of R. In particular, if X is a candidate key of R, then **X → R** (X determines *every* attribute in R).

2. **FDs are not symmetric:** If **X → Y** holds in R, this tells us **nothing** about whether **Y → X** also holds. (i.e., functional dependency is a **one-directional** constraint, not necessarily reversible.)

FDs Are a Property of Semantics, Not of a Particular State

A functional dependency is a **property of the meaning/semantics** of the attributes — **not** something that can be automatically read off from a snapshot of data. Database designers use their **understanding of the real-world meaning** of the attributes to **specify** which FDs must hold on **all** relation states (extensions) of R.

- A relation extension r(R) that **satisfies** all specified FD constraints is called a **legal relation state** (or legal extension) of R.
- Some FDs can be specified **without reference to a specific relation** — purely from real-world semantics. Example: {State, Driver_license_number} → Ssn should hold for any adult in the U.S., regardless of which relation these attributes appear in.
- **FDs can change or cease to exist over time** if real-world relationships change. Example: Zip_code → Area_code used to hold in the U.S. but is **no longer true** due to the proliferation of telephone area codes.

Example — EMP_PROJ Relation

For the EMP_PROJ relation (Ssn, Pnumber, Hours, Ename, Pname, Plocation), based on real-world semantics, the following FDs should hold:

a. Ssn → Ename

b. Pnumber → {Pname, Plocation}

c. {Ssn, Pnumber} → Hours

Meaning:

- (a) An employee's **Ssn uniquely determines** the employee's name.
- (b) A **project number uniquely determines** the project's name and location.
- (c) The **combination** of Ssn and Pnumber uniquely determines the **Hours** the employee works on that project per week.

FDs Cannot Be Inferred From a Single Data Snapshot Alone

A functional dependency is a property of the **relation schema R**, **not** of one particular legal state *r* of R. Therefore:

- An FD **cannot be automatically inferred** just by looking at one populated table — it must be **explicitly defined** by someone who understands the **semantics** of the attributes.
- However, it **is** sufficient to find **a single counter-example** (a violation) to **disprove** that an FD holds.

Illustrative Example — TEACH Relation

Consider a sample state of relation TEACH(Teacher, Course, Text). At first glance, one might guess Text → Course holds — but this **cannot be confirmed** without knowing it holds for **all possible legal states** of TEACH.

However, we **can** disprove some FDs using this data: since **'Smith'** teaches **both** 'Data Structures' and 'Database Systems', we can definitively conclude:

Teacher does NOT functionally determine Course

(i.e., Teacher → Course does **not** hold, because Smith is a counterexample.)

Another Example — Relation R(A, B, C, D)

Given a sample extension with 4 tuples:

A B C D

a1 b1 c1 d1

a1 b2 c2 d2

a2 b2 c2 d3

a3 b3 c4 d3

FDs that MAY hold (no violation observed in this extension):

B → C

C → B

{A, B} → C

{A, B} → D

$\{C, D\} \rightarrow B$

FDs that DO NOT hold (violations exist):

$A \rightarrow B$ (tuples 1 and 2 violate: same A=a1, but B differs: b1 vs b2)

$B \rightarrow A$ (tuples 2 and 3 violate: same B=b2, but A differs: a1 vs a2)

$D \rightarrow C$ (tuples 3 and 4 violate: same D=d3, but C differs: c2 vs c4)

Key exam point: Given a populated relation, we **cannot definitively confirm** an FD holds (we can only say it "may hold" based on that snapshot) unless we know the attribute semantics — but we **can definitively rule out** an FD if we find even one violating pair of tuples.

Diagrammatic Notation for FDs

FDs can be displayed graphically:

- Each FD is shown as a **horizontal line**.
- **LHS attributes** are connected to this line via **vertical lines** (no arrowhead).
- **RHS attributes** are connected via lines with an **arrowhead** pointing toward them.

The Set F of Functional Dependencies

- **F** denotes the set of functional dependencies **specified** on a relation schema R.
- Typically, the designer specifies only the FDs that are **semantically obvious**.
- However, **many additional FDs** can be **logically inferred/deduced** from the FDs in F — these hold in every legal relation instance that satisfies F. (The formal **inference rules** for deriving these are covered in **Chapter 15**.)

4.8 NORMAL FORMS BASED ON PRIMARY KEYS

Purpose

Having defined functional dependencies, we now use them to build a **formal methodology** for testing and improving relation schemas. We assume:

- A set of FDs is given for each relation.
- Each relation has a designated **primary key**.

These, combined with **normal form tests (conditions)**, **drive the normalization process**.

Two Common Approaches to Practical Relational Design

1. Perform **conceptual schema design** (using ER/EER model) and **map** it into a set of relations.
2. Design relations based on **external knowledge** from existing files, forms, or reports.

In either case, it is useful to **evaluate** the relations for quality and **decompose** them further as needed to achieve **higher normal forms**.

4.8.1 Normalization of Relations

The Normalization Process

- First proposed by **Codd (1972)**.
- Takes a relation schema through a **series of tests** to certify whether it satisfies a certain **normal form**.
- Proceeds in a **top-down** fashion: evaluate each relation against normal form criteria, and **decompose** relations as necessary.
- Considered **"relational design by analysis"**.

History of Normal Forms

Normal Form	Proposed By
1NF, 2NF, 3NF	Codd (1972)
BCNF (Boyce-Codd Normal Form)	Boyce and Codd (stronger version of 3NF)
4NF	Based on multivalued dependencies (Section 14.6)
5NF	Based on join dependencies (Section 14.7)

All these normal forms are based on a **single analytical tool**: the **functional dependencies** among the attributes of a relation.

Purpose of Normalization

Normalization analyzes relation schemas based on their **FDs and primary keys**, to achieve two desirable properties:

1. **Minimizing redundancy**
2. **Minimizing insertion, deletion, and update anomalies** (discussed in Section 14.1.2)

It can be thought of as a **"filtering" or "purification" process** — improving design quality step by step. A relation that **fails** a normal form test is **decomposed** into smaller relation schemas that **do** satisfy the test.

What Normalization Provides

1. A **formal framework** for analyzing relation schemas based on keys and FDs.
2. A **series of normal form tests** that can be applied to individual relation schemas so the database can be normalized to any desired degree.

Definition — Normal Form of a Relation

Definition: The **normal form** of a relation refers to the **highest** normal form condition that it satisfies — indicating the degree to which it has been normalized.

Important Caveat

Normal forms alone (in isolation) do NOT guarantee a good database design. It is **not sufficient** to check each relation individually for, say, BCNF or 3NF. The overall **decomposition process** must also confirm **two additional properties** across the relations taken together:

1. **Nonadditive join property (lossless join property):** Guarantees that the **spurious tuple generation** problem (Section 14.1.4) does **not** occur when the decomposed relations are joined back.
2. **Dependency preservation property:** Ensures that **every functional dependency** is represented in **some** individual relation after decomposition (i.e., no FD is "lost" and unenforceable after decomposition).

Relative importance:

- The **nonadditive join property** is **extremely critical** and **must be achieved at any cost**.
- The **dependency preservation property**, though desirable, is **sometimes sacrificed**

4.8.2 Practical Use of Normal Forms

Real-World Practice

Most practical design projects acquire designs from **existing databases**, **legacy systems**, or **existing files/forms**. Designers evaluate these existing designs by applying **normal form tests**, and carry out normalization to achieve high-quality, sustainable designs.

Practical Limits

Although higher normal forms (4NF, 5NF) exist, their **practical utility is questionable** because:

- The constraints they are based on (multivalued dependencies, join dependencies) are **rare** and **hard to detect/understand**.
- Designers/users must already know these constraints or discover them through business analysis.

Conclusion: Industry practice typically normalizes only up to **3NF, BCNF, or at most 4NF**.

Denormalization

Designers **need not** always normalize to the highest possible normal form. Relations may deliberately be left at a **lower normalization status** (e.g., 2NF) for **performance reasons** (as discussed at the end of Section 14.1.2), while accepting the corresponding **penalty of dealing with anomalies**.

Definition: Denormalization is the process of storing the **join** of higher normal form relations **as a base relation**, which is consequently in a **lower normal form**.

14.3.3 Definitions of Keys and Attributes Participating in Keys

Superkey

Definition: A **superkey** of relation schema $R = \{A_1, A_2, ..., A_n\}$ is a set of attributes $S \subseteq R$ such that **no two tuples** t_1, t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$.

Key

A **key K** is a superkey with the **additional property** that **removing any attribute** from K causes it to **no longer be a superkey**.

Difference between key and superkey: A key must be **minimal**. If $K = \{A_1, A_2, ..., A_k\}$ is a key of R, then $K - \{A_i\}$ is **not** a key of R for **any** A_i.

Example: For EMPLOYEE, {Ssn} is a **key**. But {Ssn}, {Ssn, Ename}, {Ssn, Ename, Bdate} — and any set of attributes that **includes** Ssn — are all **superkeys** (only {Ssn} alone is minimal, hence the key).

Candidate Key and Primary Key

- If a relation schema has **more than one key**, each is called a **candidate key**.
- **One** candidate key is arbitrarily chosen/designated as the **primary key**; the rest are called **secondary keys**.
- Every relation schema in a practical database **must have a primary key**.
- If no candidate key is known, the **entire relation** can be treated as a default superkey.

Example: In EMPLOYEE, {Ssn} is the **only** candidate key, so it is automatically the primary key.

Prime and Nonprime Attributes

Definition: An attribute of R is called a **prime attribute** if it is a member of **some candidate key** of R. It is called **nonprime** if it is **not** a member of **any** candidate key.

Example: In WORKS_ON(Ssn, Pnumber, Hours), both **Ssn** and **Pnumber** are **prime** attributes (they together form the primary key); **Hours** is **nonprime**.

Sequence of Normal Forms

1NF, 2NF, and 3NF were proposed by **Codd (1972)** as a **progressive sequence**: a relation moves from 1NF → 2NF → 3NF as needed. Historically:

- **2NF** and **3NF** each independently attack **different types of problems** arising from problematic FDs.

- By convention, they are tested **in sequence** — hence, **by definition, a 3NF relation already satisfies 2NF**.

4.8.3 First Normal Form (1NF)

Definition and Purpose

1NF is now considered **part of the basic definition of a relation** in the flat relational model. Historically, it was defined to **disallow**:

- **Multivalued attributes**
- **Composite attributes**
- Combinations of the above

Rule: The domain of an attribute must include only **atomic (simple, indivisible) values**, and the value of any attribute in a tuple must be a **single value** from that attribute's domain.

Hence, 1NF **disallows**:

- A **set** of values
- A **tuple** of values
- Any **combination** of both

...as an attribute value for a single tuple. In other words, **1NF disallows "relations within relations"** or relations as attribute values within tuples.

Example — Violating 1NF

Consider DEPARTMENT(Dname, Dnumber, Dmgr_ssn, Dlocations), where each department can have **multiple locations**, so Dlocations might hold a **set** of values like {Bellaire, Sugarland, Houston} for one tuple.

This is **not in 1NF** because Dlocations is **not atomic**.

Two ways to view this:

1. The domain of Dlocations contains **atomic values**, but some tuples hold a **set** of these values — in this case Dlocations is **not** functionally dependent on Dnumber.

2. The domain of Dlocations itself contains **sets of values** (nonatomic domain) — in this case, Dnumber → Dlocations **does** hold, since each set is treated as a single domain member.

Either way, this relation **violates 1NF** and does not even qualify as a relation under the standard relational model definition.

Three Techniques to Achieve 1NF

Technique 1 — Remove the violating attribute into a new relation (BEST approach):

Remove Dlocations and place it in a **separate relation** DEPT_LOCATIONS(Dnumber, Dlocation), where the primary key is the **combination** {Dnumber, Dlocation}. A distinct tuple exists for each location of each department.

- This is generally considered the **best solution**: it avoids redundancy and places **no limit** on the number of location values.

Technique 2 — Expand the primary key (introduces redundancy):

Create a separate **tuple** in the original DEPARTMENT relation for each location, with the primary key expanded to {Dnumber, Dlocation}.

- **Disadvantage:** Introduces redundancy (Dname, Dmgr_ssn repeated per location) — **rarely adopted**. If chosen, it typically gets decomposed further in later normalization steps into Technique 1's form anyway.

Technique 3 — Use a fixed number of atomic attributes:

If a **maximum number** of values is known (e.g., at most 3 locations per department), replace Dlocations with Dlocation1, Dlocation2, Dlocation3.

- **Disadvantages:**
  - Introduces **NULL values** for departments with fewer locations than the maximum.
  - Introduces **spurious ordering semantics** among the location values (not originally intended).
  - Makes querying **harder** — e.g., "List departments having 'Bellaire' as one of their locations" becomes awkward (must check all three columns).

- **Best avoided** for these reasons.

Nested Relations (Multivalued Composite Attributes)

1NF also **disallows multivalued attributes that are themselves composite** — these create **nested relations**, where each tuple contains a "relation within it."

Example:

EMP_PROJ(Ssn, Ename, {PROJS(Pnumber, Hours)})

- The **set braces { }** indicate PROJS is **multivalued**.
- The component attributes forming PROJS are listed in parentheses.
- Ssn is the **primary key** of EMP_PROJ; Pnumber is the **partial key** of the nested relation (unique *within* each tuple).

Normalizing Nested Relations into 1NF

Procedure: Remove the nested relation's attributes into a **new relation**, and **propagate the primary key** of the original relation into it. The new relation's primary key = **partial key + propagated primary key**.

Example — Decomposing EMP_PROJ:

EMP_PROJ1 (Ssn, Ename)

EMP_PROJ2 (Ssn, Pnumber, Hours)

This procedure can be applied **recursively** for relations with **multiple levels of nesting**.

Multi-level nesting example:

CANDIDATE (Ssn, Name, {JOB_HIST (Company, Highest_position, {SAL_HIST (Year, Max_sal)})})

Using the internal partial keys Company and Year, this normalizes into:

CANDIDATE_1 (Ssn, Name)

CANDIDATE_JOB_HIST (Ssn, Company, Highest_position)

CANDIDATE_SAL_HIST (Ssn, Company, Year, Max_sal)

Handling Multiple Multivalued Attributes — A Special Caution

Consider:

PERSON (Ss#, {Car_lic#}, {Phone#})

This represents a person having **multiple cars** and **multiple phones**.

Incorrect approach (Technique 2 style): Combining into a single **all-key** relation:

PERSON_IN_1NF (Ss#, Car_lic#, Phone#)

This forces **every combination** of Car_lic# and Phone# to be represented for each Ss#, introducing **artificial redundancy** and a **spurious relationship** between Car_lic# and Phone# that doesn't actually exist. (This problem is properly addressed later using **multivalued dependencies** and **4NF** — Section 14.6.)

Correct approach (Technique 1 style): Decompose into **two separate relations**:

P1 (Ss#, Car_lic#)

P2 (Ss#, Phone#)

Large Objects (BLOB/CLOB) and 1NF

Modern databases often store **images, documents, video clips, audio clips**, etc. When stored, the **entire object/file** is treated as a **single atomic value** using the SQL data types:

- **BLOB** (Binary Large Object)
- **CLOB** (Character Large Object)

Since the object is treated as **one atomic, single-valued attribute**, the relation **retains its 1NF status**.

4.8.4 Second Normal Form (2NF)

Key Concept — Full Functional Dependency

Definition: A functional dependency **X → Y** is a **full functional dependency** if removing **any** attribute A from X causes the dependency to **no longer hold** — i.e., for every A ∈ X, **(X − {A})** does **NOT** functionally determine Y.

Definition: A functional dependency **X → Y** is a **partial dependency** if **some** attribute A can be **removed** from X and the dependency **still holds** — i.e., for some A ∈ X, **(X − {A}) → Y**.

Example (EMP_PROJ):

- {Ssn, Pnumber} → Hours is a **FULL** dependency (neither Ssn → Hours nor Pnumber → Hours holds alone).
- {Ssn, Pnumber} → Ename is a **PARTIAL** dependency, because Ssn → Ename **alone** already holds (Pnumber is not needed).

Definition of 2NF

Definition: A relation schema R is in **2NF** if **every nonprime attribute A in R is fully functionally dependent** on the **primary key** of R.

Testing for 2NF

- The test involves checking FDs whose **LHS is part of** (a proper subset of) the primary key.
- **If the primary key is a single attribute**, the 2NF test is **automatically satisfied** — **no partial dependency is possible** on a single-attribute key.

Example — EMP_PROJ Violates 2NF

EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation) with primary key {Ssn, Pnumber} and FDs:

FD1: {Ssn, Pnumber} → Hours (full — OK)

FD2: Ssn → Ename (partial — VIOLATES 2NF)

FD3: Pnumber → {Pname, Plocation} (partial — VIOLATES 2NF)

- Ename violates 2NF (via FD2) because it's determined by **only Ssn**, a **part** of the primary key.
- Pname and Plocation violate 2NF (via FD3) because they're determined by **only Pnumber**, also a **part** of the primary key.

Normalizing into 2NF

Decompose EMP_PROJ into **three** relations, each holding a nonprime attribute set together with the part of the key it fully depends on:

EP1 (Ssn, Pnumber, Hours) — based on FD1

EP2 (Ssn, Ename) — based on FD2

EP3 (Pnumber, Pname, Plocation) — based on FD3

Each of **EP1, EP2, EP3** is now in **2NF**.

4.8.5 Third Normal Form (3NF)

Key Concept — Transitive Dependency

Definition: A functional dependency **X → Y** in relation schema R is a **transitive dependency** if there exists a set of attributes **Z** in R such that:

- Z is **neither a candidate key** nor a **subset of any key** of R, **and**
- Both **X → Z** and **Z → Y** hold.

Example (EMP_DEPT): Ssn → Dmgr_ssn is **transitive** through Dnumber, because:

Ssn → Dnumber (holds)

Dnumber → Dmgr_ssn (holds)

and Dnumber is **neither a key** nor part of the key of EMP_DEPT.

Intuition: It is undesirable for Dmgr_ssn to depend on Dnumber **within** the EMP_DEPT relation, since Dnumber is **not a key** of EMP_DEPT — this dependency really "belongs" to a separate DEPARTMENT entity.

Definition of 3NF (Codd's Original)

Definition: A relation schema R is in **3NF** if it satisfies **2NF**, and **no nonprime attribute of R is transitively dependent** on the primary key.

Example — EMP_DEPT Violates 3NF

EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn) with primary key Ssn:

- **Satisfies 2NF** (primary key is a single attribute Ssn, so no partial dependency is possible).
- **Violates 3NF** because of the **transitive dependency** of Dmgr_ssn (and also Dname) on Ssn, **via** Dnumber.

Normalizing into 3NF

Decompose EMP_DEPT into **two** 3NF relations:

ED1 (Ssn, Ename, Bdate, Address, Dnumber)

ED2 (Dnumber, Dname, Dmgr_ssn)

Intuition: ED1 and ED2 represent **independent facts** about **employees** and **departments** respectively — two distinct entities in their own right. A **NATURAL JOIN** on ED1 and ED2 (via Dnumber, which is a proper primary-key/foreign-key pair) **recovers the original EMP_DEPT relation without generating spurious tuples**.

General Insight — Problematic FDs

Intuitively: **any FD whose LHS is a proper subset of the primary key** (→ violates 2NF), or **any FD whose LHS is a nonkey attribute** (→ violates 3NF), is a **"problematic" FD**.

2NF and 3NF normalization work by **decomposing** the original relation to **eliminate** these problematic FDs.

Order of Testing

- It is **not strictly necessary** to remove partial dependencies before transitive dependencies.
- However, **historically**, 3NF has been defined assuming a relation is **tested for 2NF first**, then for 3NF.
- The more **general definition** of 3NF (given later in Section 14.4.2) **automatically incorporates** the 2NF condition as well.

First Three Normal Forms

Normal Form	Based On	Condition	Violation Symptom	Remedy
1NF	Atomicity of attribute values	All attribute values must be **atomic** (single, indivisible)	Multivalued or composite attributes; nested relations	Decompose: move multivalued attribute into a new relation with propagated key

Normal Form	Based On	Condition	Violation Symptom	Remedy
2NF	Full functional dependency	Every **nonprime** attribute must be **fully** dependent on the **entire** primary key	**Partial dependency**: nonprime attribute depends on only *part* of a composite primary key	Decompose relation so each nonprime attribute group is with the key part it fully depends on
3NF	Transitive dependency	Relation must satisfy 2NF, **and** no nonprime attribute is **transitively** dependent on the primary key	**Transitive dependency**: nonprime attribute depends on another nonprime attribute (which depends on the key)	Decompose relation to remove the transitive chain into a separate relation

4.9 General Definitions of Second and Third Normal Forms

Introduction — Why We Need More General Definitions

The goal in relational design is to eliminate **both partial and transitive dependencies**, since these are the root cause of the **update anomalies** discussed in Section 14.1.2.

The definitions of **2NF and 3NF** given earlier (Section 14.3) were based **only on the primary key**. These earlier definitions are useful for **practical analysis** where the primary key has already been chosen for a given database, but they have a limitation:

They ignore other candidate keys, if any exist, in the relation.

This section provides the **more general definitions** of **2NF and 3NF** that take **all candidate keys** of a relation into account — not just the primary key.

Important Clarifications

- **1NF is unaffected** by this generalization, because 1NF is **independent of keys and functional dependencies** altogether (it deals purely with attribute atomicity).

- **General definition of "prime attribute":** An attribute is **prime** if it is part of **any** candidate key (not just the primary key).
- **Partial dependency, full dependency, and transitive dependency** must now be evaluated **with respect to every candidate key** of the relation, not just the primary key.

4.9.1 General Definition of Second Normal Form

Definition

Definition: A relation schema R is in **second normal form (2NF)** if **every nonprime attribute A in R is not partially dependent on any key of R**.

Alternative phrasing: A relation schema R is in 2NF if every nonprime attribute A in R is **fully functionally dependent** on **every** key of R (not just the primary key).

Testing for 2NF

- The test checks FDs whose **LHS attributes are part of a key**.
- If a key contains only a **single attribute**, the test is **automatically satisfied** for that key.

Illustrative Example — The LOTS Relation

Consider LOTS(Property_id#, County_name, Lot#, Area, Price, Tax_rate), describing land parcels for sale across counties.

Two candidate keys exist:

1. Property_id# (unique statewide)
2. {County_name, Lot#} (lot numbers are unique **only within** each county)

Primary key chosen: Property_id# (underlined by convention) — but note that **no special preference** is given to this key over the other candidate key when testing normal forms; **both** candidate keys must be considered.

Given functional dependencies:

FD1: Property_id# → {County_name, Lot#, Area, Price, Tax_rate}

FD2: {County_name, Lot#} → {Property_id#, Area, Price, Tax_rate}

FD3: County_name → Tax_rate

FD4: Area → Price

Meaning:

- **FD1 & FD2** — Express that both candidate keys determine all other attributes (as expected for a key).
- **FD3** — The **tax rate is fixed for a given county** (doesn't vary lot-by-lot within the same county).
- **FD4** — The **price of a lot is determined by its area**, regardless of which county it's in (assume this is the price for tax purposes).

Why LOTS Violates the General 2NF

LOTS violates the **general definition of 2NF** because:

Tax_rate is **partially dependent** on the candidate key {County_name, Lot#}, due to **FD3** (County_name → Tax_rate).

Here, County_name is only **part of** the candidate key {County_name, Lot#} — so Tax_rate depends on just a **portion** of this key, which is a **partial dependency**, violating 2NF.

*(Note: this problem would NOT be caught by the primary-key-only definition, since the primary key is Property_id#, a single attribute — the violation only becomes visible when we also check the **second candidate key**.)*

Normalizing LOTS into 2NF

Decompose LOTS into **two relations**:

LOTS1 (Property_id#, County_name, Lot#, Area, Price)

LOTS2 (County_name, Tax_rate)

Method: Remove the violating attribute Tax_rate from LOTS, and place it together with County_name (the **LHS** of FD3, which causes the partial dependency) into a new relation LOTS2.

- **FD4** (Area → Price) does **not** violate 2NF (since Area is not part of either candidate key), so it is **carried over unchanged** into LOTS1.

- **Both LOTS1 and LOTS2 are now in 2NF.**

4.9.2 General Definition of Third Normal Form

Definition

Definition: A relation schema R is in **third normal form (3NF)** if, whenever a **nontrivial functional dependency X → A** holds in R, **either**:

- **(a)** X is a **superkey** of R, **or**
- **(b)** A is a **prime attribute** of R.

This is the **general** definition — it accounts for **all candidate keys**, not just the primary key, and it applies to **any** nontrivial FD in the relation, not only FDs involving the key.

Applying the Definition to LOTS1

Recall LOTS1(Property_id#, County_name, Lot#, Area, Price) with FD4: Area → Price.

Check FD4 against the 3NF conditions:

- (a) Is Area a superkey of LOTS1? **No.**
- (b) Is Price a prime attribute of LOTS1? **No.**

Since **neither** condition holds, **FD4 violates the general 3NF definition** — LOTS1 is **NOT in 3NF**.

(Note: LOTS2, from the 2NF decomposition, **is already in 3NF** — it has no such problematic FD.)

Normalizing LOTS1 into 3NF

Decompose LOTS1 into:

LOTS1A (Property_id#, County_name, Lot#, Area)

LOTS1B (Area, Price)

Method: Remove Price (the attribute violating 3NF) from LOTS1, and place it together with Area (the **LHS** of FD4, causing the **transitive dependency**) into a new relation LOTS1B.

- **Both LOTS1A and LOTS1B are now in 3NF.**

Overall Decomposition Path

LOTS (1NF)

└── LOTS1, LOTS2 (2NF)

└── LOTS1A, LOTS1B (3NF) [from LOTS1]

LOTS2 (already 3NF)

Final 3NF schema: **LOTS1A, LOTS1B, LOTS2**

Two Important Observations

1. **Why LOTS1 violates 3NF, restated intuitively:** Price is **transitively dependent** on **each of the candidate keys** of LOTS1, **via** the nonprime attribute Area. That is: {Property_id#} → Area → Price (and similarly through the other candidate key) — a classic transitive dependency chain.
2. **The general 3NF definition can be applied directly — no need to test 2NF first.** This is a major practical advantage:

If a relation schema **passes** the general 3NF test, it **automatically** passes the 2NF test as well.

This means the **order** in which we remove partial vs. transitive dependencies **does not matter** — applying the general 3NF definition directly to the **original** LOTS relation (with FD1–FD4) would immediately reveal that **both FD3 and FD4 violate 3NF**:

- **FD3** (County_name → Tax_rate): County_name is **not a superkey** of LOTS → violates condition (a); Tax_rate is not prime → violates (b) too.
- **FD4** (Area → Price): Same reasoning — violates 3NF directly.

So we could decompose LOTS **directly** into LOTS1A, LOTS1B, and LOTS2 in **one pass**, without going through the intermediate 2NF step explicitly.

4.9.3 Interpreting the General Definition of Third Normal Form

Breaking Down "Condition (a)" of the 3NF Definition

A relation schema R **violates** the general 3NF definition if there exists a functional dependency **X → A** where **neither (a) nor (b)** holds. Let's look closely at what **condition (a) alone** (X is a superkey) is designed to "catch." Condition (a) failing captures **two distinct types of problematic dependencies**:

1. **A nonprime attribute determines another nonprime attribute.** → This is essentially a **transitive dependency**, which violates 3NF (as in the classic Section 14.3.6 sense).
2. **A proper subset of a key functionally determines a nonprime attribute.** → This is a **partial dependency**, which violates 2NF (as in Section 14.3.5/14.4.1).

Conclusion: Condition (a) **by itself** already addresses **both** of the problematic dependency types that originally motivated the separate definitions of 2NF and 3NF. This confirms why testing the general 3NF definition **subsumes** the 2NF test.

Alternative (Equivalent) Definition of 3NF

Based on this insight, we can restate 3NF as follows:

Alternative Definition: A relation schema R is in **3NF** if **every nonprime attribute** of R satisfies **both** of the following conditions:

1. It is **fully functionally dependent** on **every key** of R.
2. It is **nontransitively dependent** on **every key** of R.

This alternative definition makes the **connection to 2NF and the classic 3NF concept (Section 14.3)** more explicit — condition 1 rules out partial dependencies (2NF violations), and condition 2 rules out transitive dependencies (3NF violations in the original sense).

The Loophole in the General 3NF Definition — Condition (b)

Here's a subtle but **important** point for exams:

Condition (b) of the general 3NF definition — "A is a **prime** attribute" — allows **certain functional dependencies to "slip through" or escape** the 3NF test.

That is: even if X → A holds where **X is NOT a superkey** (which would normally indicate a problem), the dependency is **still considered acceptable under 3NF** as long as **A happens to be a prime attribute** (part of some candidate key).

This means: 3NF **does not catch every potentially problematic dependency** — some dependencies where the determinant (X) is not a superkey are still allowed to exist, **purely because** the determined attribute (A) is prime.

This is precisely the gap that Boyce-Codd Normal Form (BCNF) closes:

BCNF is a stricter normal form that **removes** this loophole — it does **not** allow such dependencies to "escape," **regardless of whether A is prime or not**. BCNF requires that the **determinant (X)** always be a **superkey**, full stop.

4.10 Boyce-Codd Normal Form (BCNF)

Introduction

Boyce-Codd Normal Form (BCNF) was originally proposed as a **simpler** reformulation of 3NF, but it turned out to be **stricter** than 3NF. This gives us an important relationship:

Every relation in BCNF is also in 3NF.
But a relation in 3NF is NOT necessarily in BCNF.

As noted in Section 14.4.3: although **3NF allows** functional dependencies that satisfy clause **(b)** of its general definition (i.e., the RHS attribute is prime), **BCNF disallows such dependencies entirely** — making it a **stronger (stricter)** normal form.

Motivating Example — Why We Need BCNF

Recall the LOTS1A(Property_id#, County_name, Lot#, Area) relation from Section 14.4, with:

FD1: Property_id# → {County_name, Lot#, Area}

FD2: {County_name, Lot#} → {Property_id#, Area}

Introducing a New Real-World Constraint

Suppose there are **thousands of lots**, but they come from only **two counties**: **DeKalb** and **Fulton**. Suppose further that:

- Lot sizes in **DeKalb** County are restricted to: 0.5, 0.6, 0.7, 0.8, 0.9, 1.0 acres
- Lot sizes in **Fulton** County are restricted to: 1.1, 1.2, ..., 1.9, 2.0 acres

Since the **area value alone** now determines which county a lot belongs to, this introduces a new functional dependency:

FD5: Area → County_name

Why LOTS1A Is Still (Technically) in 3NF

Adding FD5 does **not** cause LOTS1A to violate 3NF. Why? Because under the **general 3NF definition** (Section 14.4.2), a violation requires that **both**:

- (a) X is **not** a superkey, **AND**
- (b) A is **not** a prime attribute

Here, X = Area is indeed **not** a superkey — but County_name **is a prime attribute** (it's part of the candidate key {County_name, Lot#}). So **condition (b) is satisfied**, and FD5 **"slips through"** the 3NF test — exactly the loophole described in Section 14.4.3.

But There's a Real Redundancy Problem

Even though LOTS1A technically satisfies 3NF, FD5 still causes **unnecessary redundancy**: since there are only **16 possible Area values** (6 for DeKalb + 10 for Fulton), the mapping Area → County_name could be represented compactly in a **separate relation** R(Area, County_name) with just **16 tuples**.

Instead, storing County_name directly in LOTS1A means this same fact (e.g., "Area 0.5 → DeKalb") gets **repeated across thousands of LOTS1A tuples** — one for every lot, causing exactly the kind of **redundancy and update anomaly risk** that normalization is meant to eliminate.

Conclusion: This shows the **need for a normal form stronger than 3NF** — which is exactly what **BCNF** provides.

Formal Definition of BCNF

Definition: A relation schema R is in **BCNF** if, whenever a **nontrivial functional dependency X → A** holds in R, then **X is a superkey** of R.

Comparison with the General 3NF Definition

Recall the general 3NF definition (Section 14.4.2):

R is in 3NF if for every nontrivial FD X → A: **(a) X is a superkey, OR (b) A is a prime attribute.**

BCNF removes clause (b) entirely. The BCNF definition requires **only** that X be a superkey — there is **no escape clause** for the case where A is prime.

	3NF	BCNF
Condition for X → A to be acceptable	X is a superkey, **OR** A is prime	X is a superkey (**only**)
Stricter/Weaker	Weaker (allows more relations to pass)	Stricter (fewer relations pass)

This makes **BCNF a strictly stronger (more restrictive) condition** than 3NF.

Applying BCNF to the LOTS1A Example

In LOTS1A, the dependency FD5: Area → County_name **violates BCNF** because Area is **not a superkey** of LOTS1A (regardless of the fact that County_name is prime).

Decomposing LOTS1A into BCNF

We decompose LOTS1A into two BCNF relations:

LOTS1AX (Property_id#, Area, Lot#)

LOTS1AY (Area, County_name)

Important consequence — loss of a functional dependency:

This decomposition **loses FD2** ({County_name, Lot#} → {Property_id#, Area}), because the attributes County_name and Lot# **no longer coexist together** in the same relation after decomposition. This is an example of **sacrificing the dependency preservation property** in order to eliminate redundancy — a trade-off discussed generally in Section 14.3.1 (recall: the **nonadditive join property is mandatory**, but **dependency preservation may sometimes be sacrificed**).

General Insight — When Does 3NF ≠ BCNF?

In **practice**, most relation schemas that satisfy 3NF **also** satisfy BCNF.

A relation R is in **3NF but NOT BCNF only if** there exists **some FD X → A** in R such that:

- **X is NOT a superkey**, **AND**
- **A IS a prime attribute**

This is precisely the **"escape clause" scenario** — illustrated generically by the schematic relation R(A, B, C) with FDs AB → C (or similar) and C → B, where C is not

a superkey but B is prime. Such an FD leads to **potential data redundancy**, exactly as demonstrated by FD5: Area → County_name in LOTS1A.

Design Recommendation

Ideally, relational database design should strive to achieve BCNF (or at least 3NF) for every relation schema.

Achieving **only 1NF or 2NF is NOT considered adequate**, because both were developed **historically as intermediate stepping stones** on the path toward 3NF and BCNF — they are not, by themselves, satisfactory end goals for a well-designed schema.

4.10.1 Decomposition of Relations Not in BCNF

Example — The TEACH Relation

Consider the relation TEACH(Student, Course, Instructor) with the functional dependencies:

FD1: {Student, Course} → Instructor

FD2: Instructor → Course

Meaning of FD2: Each instructor teaches **only one course** — this is a real-world constraint of this particular application (e.g., a small department where instructors are course-specific).

Why TEACH Is in 3NF but NOT BCNF

- {Student, Course} is a **candidate key** for this relation.
- The FD pattern here matches the general "problematic-but-3NF-compliant" schematic relation from Figure 14.13(b): if we map **Student = A**, **Course = B**, **Instructor = C**, then the pattern is AB → C and C → B.
- Since Instructor (the LHS of FD2) is **not a superkey**, but Course (its RHS) **is a prime attribute** (part of the candidate key {Student, Course}) — FD2 satisfies **3NF's clause (b) loophole**.
- Hence: **TEACH is in 3NF, but violates BCNF** (because of FD2, where Instructor is not a superkey).

The Challenge of Decomposing TEACH

Unlike the LOTS1A example, decomposing TEACH into BCNF relations is **not straightforward**, because there are **three possible ways** to split it into two relations:

Option 1: R1(Student, Instructor) and R2(Student, Course)

Option 2: R1(Course, Instructor) and R2(Course, Student)

Option 3: R1(Instructor, Course) and R2(Instructor, Student)

Common problem: All three decompositions **lose FD1** ({Student, Course} → Instructor) — none of them can preserve this dependency, since Student, Course, and Instructor never all appear together in one relation after any binary split.

Key question: Since dependency preservation is unavoidable to lose here, **which decomposition is still acceptable?** The answer depends on whether it satisfies the **mandatory** nonadditive (lossless) join property.

The Nonadditive Join Test for Binary Decompositions (NJB Test)

Recall from Section 14.3.1: while **dependency preservation** may sometimes be sacrificed, the **nonadditive join property is mandatory** and must **always** be satisfied.

NJB Test: A decomposition **D = {R1, R2}** of relation R has the **lossless (nonadditive) join property** with respect to a set of FDs F on R **if and only if EITHER**:

- The FD **(R1 ∩ R2) → (R1 − R2)** is in **F^+**, **OR**
- The FD **(R1 ∩ R2) → (R2 − R1)** is in **F^+**

*(F^+ denotes the **closure** of F — the set of **all** FDs that are logically implied by F. This is discussed in detail in Chapter 15, Section 15.1. For our purposes here, it's enough to check whether **one of the two FDs above actually holds**.)*

Applying the NJB Test to the Three TEACH Decompositions

Testing Option 3: R1(Instructor, Course) and R2(Instructor, Student)

- **R1 ∩ R2** = {Instructor}
- **R1 − R2** = {Course}
- Check: Does Instructor → Course hold? **YES** (this is exactly FD2).

The NJB test is satisfied — Option 3 has the **nonadditive join property**.

*(It is left as an exercise to verify that Options 1 and 2 do **NOT** satisfy the NJB test — i.e., neither of the two required FDs holds for those splits.)*

Correct BCNF Decomposition of TEACH

TEACH1 (Instructor, Course)

TEACH2 (Instructor, Student)

This is the **proper** decomposition of TEACH into BCNF relations, since it is the **only** option among the three that guarantees a **lossless join** back to the original relation.

General Algorithm — Decomposing a Relation into BCNF

A relation **R not in BCNF** can be systematically decomposed (while preserving the nonadditive join property) using the following recursive procedure:

Procedure:

1. Let **R** be the relation **not in BCNF**.
2. Let **X ⊆ R**, and let **X → A** be the FD that **causes the BCNF violation** (i.e., X is not a superkey).
3. Decompose **R** into **two** relations:
    - **R − A** (all attributes of R except A)
    - **XA** (the attributes of X together with A)
4. If **either** resulting relation (**R − A** or **XA**) is **still not in BCNF**, **repeat** the process on that relation.

*(This procedure is based on **Algorithm 15.5** from Chapter 15, used for producing BCNF schemas by decomposing a universal relation schema.)*

Verifying the Algorithm on Our Examples

- **Applying to LOTS1A** (violating FD: Area → County_name, so X = Area, A = County_name):
    - R − A = LOTS1A − County_name = (Property_id#, Lot#, Area) → this is **LOTS1AX**
    - XA = (Area, County_name) → this is **LOTS1AY**
    - Matches the earlier decomposition exactly.

- **Applying to TEACH** (violating FD: Instructor → Course, so X = Instructor, A = Course):
  - R − A = TEACH − Course = (Student, Instructor) → this is **TEACH2**
  - XA = (Instructor, Course) → this is **TEACH1**
  - Matches the earlier decomposition exactly.

An Important Observation — Alternate Paths to the Same BCNF Design

If we designate {Student, Instructor} as the **primary key** of TEACH, then the FD Instructor → Course causes a **partial (non-fully-functional) dependency** of Course on **part of** this key (Instructor alone, not the full key).

This means:

- This FD **could also be removed** via **ordinary second normalization** (2NF), **or**
- Via **direct application** of the general BCNF decomposition procedure above.

Both approaches yield exactly the same two resulting relations (TEACH1 and TEACH2). This illustrates an important general principle:

The same ultimate BCNF design can sometimes be reached via alternative/different paths of normalization (e.g., going through 2NF first vs. applying BCNF decomposition directly) — the end result is consistent regardless of the route taken.

4.11 Multivalued Dependency and Fourth Normal Form

Introduction — The Motivating Problem

So far, our normal forms (1NF through BCNF) have all been based on **functional dependencies (FDs)**. But there exist relations that are **already in BCNF** and yet still suffer from **redundancy** — because the problem they exhibit **cannot be expressed as a functional dependency at all**. This section introduces a **new type of constraint** — the **multivalued dependency (MVD)** — to formally capture and eliminate this kind of redundancy, leading to **Fourth Normal Form (4NF)**.

Illustrative Example — The EMP Relation

Consider the relation EMP(Ename, Pname, Dname). A tuple represents: *employee Ename works on project Pname AND has a dependent named Dname.*

Key real-world facts:

- An employee may work on **several projects**.
- An employee may have **several dependents**.
- **Crucially: the employee's projects and dependents are completely independent of one another** (a project has no relationship to a dependent).

*(In an ER diagram, each of these — projects and dependents — would naturally be modeled as a **multivalued attribute** or a **weak entity type**.)*

Sample Data — Why Redundancy Occurs

Suppose employee **Smith** works on projects **'X'** and **'Y'**, and has dependents **'John'** and **'Anna'**.

Since projects and dependents are independent, and we must **avoid implying any false relationship** between a specific project and a specific dependent, the relation must contain a tuple for **every combination**:

Ename	Pname	Dname
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

This gives **4 tuples** (2 projects × 2 dependents) just to represent 2 independent facts about one employee — this is a **combinatorial explosion of redundancy**.

Why This Redundancy Is NOT Caught by BCNF

- EMP is an **all-key relation** — its only key is {Ename, Pname, Dname} (all attributes together), since **no proper subset** of attributes functionally determines the rest.
- Since **no functional dependency** holds at all in EMP (other than trivial ones), the relation **trivially satisfies BCNF** (there's no FD X → A with X not a superkey, because there are no nontrivial FDs to begin with!).

- **Yet the redundancy is obvious and undesirable**: Pname values are repeated for every Dname, and vice versa.

Conclusion: This shows that **BCNF is not sufficient** to catch all forms of redundancy — we need a **new kind of dependency** (beyond FDs) to formally describe and eliminate this problem.

General Pattern

Whenever **two independent 1:N (or M:N) relationships**, say **A:B** and **A:C**, are **mixed together** in the same relation R(A, B, C), a **multivalued dependency (MVD)** may arise.

This MVD is denoted: **A →→ B | C**

4.11.1 Formal Definition of Multivalued Dependency

Formal Definition

Definition: A multivalued dependency **X →→ Y**, specified on relation schema R (where X, Y ⊆ R), specifies the following constraint on **any** relation state r of R:

If two tuples **t1** and **t2** exist in r such that **t1[X] = t2[X]**, then **two tuples t3 and t4** must also exist in r, with (letting **Z = R − (X ∪ Y)**):

- **t3[X] = t4[X] = t1[X] = t2[X]**
- **t3[Y] = t1[Y]** and **t4[Y] = t2[Y]**
- **t3[Z] = t2[Z]** and **t4[Z] = t1[Z]**

*(Note: t1, t2, t3, t4 are **not necessarily distinct** tuples — in some cases t3 or t4 may coincide with t1 or t2.)*

Intuitive Meaning

Whenever **X →→ Y** holds, we say **"X multidetermines Y."**

Key symmetry property: Because of the symmetric structure of the definition, whenever **X →→ Y** holds in R, it is **automatically also true** that **X →→ Z** holds (where Z is the remaining attributes). This is why an MVD is often written using the combined notation:

X →→ Y | Z

Trivial vs. Nontrivial MVDs

An MVD **X** →→ **Y** in R is called a **trivial MVD** if either: **(a)** Y is a **subset** of X, **or** **(b)** X ∪ Y = R (i.e., X and Y together cover all attributes of R)

Example: In EMP_PROJECTS(Ename, Pname), the MVD Ename →→ Pname is **trivial** (since X ∪ Y = {Ename, Pname} = R). Similarly, EMP_DEPENDENTS has the trivial MVD Ename →→ Dname.

An MVD that satisfies **neither (a) nor (b)** is called a **nontrivial MVD**.

Important property: A **trivial MVD** holds in **every** relation state r of R automatically — it is called "trivial" precisely because it **does not impose any real/meaningful constraint** on the relation. In contrast, a **nontrivial MVD** represents a genuine constraint that can cause redundancy.

The Redundancy Caused by Nontrivial MVDs

In the EMP relation (Figure 14.15a), the nontrivial MVDs are:

Ename →→ Pname

Ename →→ Dname

Because of these, the values **'X'** and **'Y'** (of Pname) must be **repeated** for **each** value of Dname — and symmetrically, **'John'** and **'Anna'** (of Dname) must be repeated for **each** value of Pname. This redundancy is clearly **undesirable**.

Yet, as established earlier, EMP is in BCNF (since no FDs hold in it at all). This demonstrates the need for a **stronger normal form** — one based on MVDs rather than FDs — to catch and eliminate this kind of problem.

Key Observations About Relations With Nontrivial MVDs

- Relations containing **nontrivial MVDs** tend to be **all-key relations** — i.e., their primary key consists of **all** the attributes taken together.
- It is **rare in practice** for such all-key relations, with this kind of **combinatorial repetition of values**, to be intentionally designed — but **recognizing MVDs as a potential problem** is still an **essential skill** in relational database design.

Definition of Fourth Normal Form (4NF)

Definition: A relation schema R is in **4NF** with respect to a set of dependencies **F** (which includes **both** functional dependencies and multivalued dependencies) if, for **every nontrivial multivalued dependency X →→ Y** in **F⁺**, **X is a superkey** for R.

*(**F⁺** denotes the **closure** of F — i.e., the set of **all** dependencies, both FDs and MVDs, that are logically **implied by** F. Formally defined in Section 15.1.)*

Key Points About 4NF

1. **An all-key relation is always in BCNF**, since (by definition of "all-key") it has **no nontrivial FDs** to violate BCNF's condition.
2. **However, an all-key relation can still violate 4NF** if it has a **nontrivial MVD** where the LHS is **not** a superkey.
    - Example: The EMP relation has **no FDs**, but **does** have the nontrivial MVD Ename →→ Pname | Dname, and Ename **alone is not a superkey** of EMP (recall, the key is the *entire* attribute set {Ename, Pname, Dname}). Hence, **EMP is NOT in 4NF**, despite being in BCNF.
3. A relation that is **not** in 4NF (due to a nontrivial MVD) **must be decomposed** to convert it into a set of relations that **are** in 4NF.
4. **This decomposition removes the redundancy** that was caused by the MVD.

Relationship Between BCNF and 4NF

4NF is stronger than BCNF — every relation in 4NF is also in BCNF, but not vice versa (as shown by the EMP counterexample: it's in BCNF but not 4NF).

This parallels the earlier relationship: **BCNF ⊆ 3NF** and now, **4NF ⊆ BCNF**.

Normalization Process — Decomposing into 4NF

General Method

To normalize a relation containing nontrivial MVDs that is not in 4NF, **decompose it** so that **each MVD is represented by a separate relation**, where — within that new relation — the MVD becomes a **trivial** MVD (and therefore no longer causes a problem).

Worked Example — Decomposing EMP

EMP(Ename, Pname, Dname) is **not in 4NF** because:

- The nontrivial MVDs Ename →→ Pname and Ename →→ Dname both hold.
- Ename is **not a superkey** of EMP (the key is the full attribute set).

Decomposition:

EMP_PROJECTS (Ename, Pname)

EMP_DEPENDENTS (Ename, Dname)

Verification that both resulting relations are in 4NF:

- In EMP_PROJECTS, the MVD Ename →→ Pname is now **trivial** (since X ∪ Y = {Ename, Pname} = the entire relation) — no violation.
- In EMP_DEPENDENTS, the MVD Ename →→ Dname is similarly **trivial**.
- **No other nontrivial MVDs** hold in either resulting relation.
- **No FDs** hold in either relation either.

Therefore, **both EMP_PROJECTS and EMP_DEPENDENTS are in 4NF.**

Why This Decomposition Makes Sense

By separating the two **independent** multivalued facts (which projects an employee works on, and which dependents an employee has) into **two separate relations**, we:

- Eliminate the artificial combinatorial pairing of unrelated facts (Project X isn't tied to dependent John in any real sense).
- Each employee's projects are now listed **once each** in EMP_PROJECTS (no repetition needed per dependent).
- Each employee's dependents are now listed **once each** in EMP_DEPENDENTS (no repetition needed per project).

This is the standard fix for the classic **"two independent multivalued attributes combined into one relation"** design flaw.

4.12 Join Dependencies and Fifth Normal Form

Introduction — Recap and Motivation

So far, the normalization process (1NF → 2NF → 3NF → BCNF → 4NF) has proceeded by identifying **problematic dependencies** (functional dependencies and multivalued

dependencies) and eliminating them through **repeated binary decomposition** — i.e., splitting one relation into **two** relations at each step.

Key constraint on binary decompositions: Every such binary split must satisfy the **NJB (Nonadditive Join for Binary decomposition) property** introduced in Section 14.5, to guarantee no spurious tuples are generated when the relations are joined back.

Achieving **4NF** typically also involves eliminating MVDs through **repeated binary decomposition**.

The Gap That Motivates 5NF

However, there exist certain relations where:

- There is **no** nonadditive (lossless) join decomposition into **two** relation schemas but there **IS** a nonadditive join decomposition into **more than two** relation schemas (i.e., a **multiway** decomposition).
- Furthermore, such a relation may have:
    - **No functional dependency** that violates any normal form up to BCNF, **and**
    - **No nontrivial MVD** that violates 4NF.

In such cases, we need a **new, more general** type of dependency: the **join dependency (JD)**. When a JD exists, we may need to carry out a **multiway decomposition** to achieve **Fifth Normal Form (5NF)**.

Important practical note: A join dependency is a **peculiar semantic constraint** that is **very difficult to detect in practice**. As a result, **normalization into 5NF is rarely performed** in real-world database design projects.

Formal Definition of Join Dependency (JD)

Definition: A **join dependency (JD)**, denoted **JD(R1, R2, ..., Rn)**, specified on relation schema R, specifies a constraint on the states *r* of R. The constraint states that **every legal state r of R** must have a **nonadditive join decomposition** into R1, R2, ..., Rn. That is, for every such r:

* (πR1(r), πR2(r), ..., πRn(r)) = r

(i.e., projecting r onto each Ri and then **natural-joining all the projections back together** reproduces **exactly** r — no more, no fewer tuples.)

Relationship Between MVD and JD

An MVD is a special case of a JD where n = 2.

That is: a JD denoted **JD(R1, R2)** (with only **two** components) is **equivalent to** the MVD:

$(R1 \cap R2) \twoheadrightarrow (R1 - R2)$

(or, by the symmetry property of MVDs discussed in Section 14.6, equivalently $(R1 \cap R2) \twoheadrightarrow (R2 - R1)$).

In other words: JD is a **generalization** of MVD — MVD handles the 2-way (binary) decomposition case, while JD handles the **general n-way** decomposition case.

Trivial vs. Nontrivial Join Dependency

A join dependency **JD(R1, R2, ..., Rn)**, specified on relation schema R, is a **trivial JD** if **one of the Ri** in the JD is **equal to R itself**.

Such a JD is called "trivial" because it has the **nonadditive join property automatically**, for **any** relation state r of R so it doesn't actually specify any meaningful constraint on R (similar in spirit to trivial FDs and trivial MVDs).

Definition of Fifth Normal Form (5NF)

Definition: A relation schema R is in **fifth normal form (5NF)** also called **project-join normal form (PJNF)** with respect to a set F of functional, multivalued, and join dependencies, if **for every nontrivial join dependency JD(R1, R2, ..., Rn) in F^+** (i.e., implied by F), **every Ri is a superkey of R**.

Illustrative Example — The SUPPLY Relation

Consider the all-key relation SUPPLY(Sname, Part_name, Proj_name) — representing which **supplier** supplies which **part** to which **project**.

The Additional Real-World Constraint

Suppose the following business rule **always holds**:

Whenever supplier **s** supplies part **p**, AND project **j** uses part **p**, AND supplier **s** supplies **at least one** part to project **j** — **then** supplier **s** will **also** be supplying part **p** to project **j**.

This is a **three-way interaction constraint** among suppliers, parts, and projects — it **cannot** be expressed as a simple FD or MVD; it requires a **join dependency**.

This constraint corresponds to the join dependency:

JD(R1, R2, R3)

where:

R1 (Sname, Part_name)

R2 (Sname, Proj_name)

R3 (Part_name, Proj_name)

These are the **three binary projections** of SUPPLY.

What This Means for the Data

If this JD constraint holds, then **whenever** the tuples "above the dashed line" (in the textbook's Figure 14.15c) exist in SUPPLY, the **additional tuples "below the dashed line"** are **logically forced** to also exist, in order to satisfy the constraint — otherwise the relation state would violate the business rule.

Decomposing SUPPLY into 5NF

SUPPLY is decomposed into **three** relations, each being a **projection**:

R1 (Sname, Part_name)

R2 (Sname, Proj_name)

R3 (Part_name, Proj_name)

Each of **R1, R2, R3** is in **5NF**.

Critical Property — Why All Three Are Needed Together

Applying a **natural join to any TWO** of these three relations **produces spurious tuples**. But applying a natural join to **ALL THREE together** does **NOT** produce spurious tuples — it correctly reconstructs the original SUPPLY relation.

Why this happens: Because **only the JD exists** here — **no MVDs are present** in SUPPLY. If an MVD existed, a **binary** decomposition would have sufficed (as in Section 14.6). But since the constraint is a genuine **3-way join dependency** (not reducible to any

pairwise MVD), **all three projections are required simultaneously** to correctly reconstruct the original data without introducing spurious rows.

Practical Difficulty of Detecting Join Dependencies

Discovering JDs in practical databases with hundreds of attributes is next to impossible. It can only be done with a **great degree of intuition** about the data on the part of the database designer.

Consequence: Current database design practice **pays scant (very little) attention** to join dependencies and 5NF, precisely because they are so hard to detect and verify in real-world scenarios.

A Useful Theoretical Shortcut — Date and Fagin's Result (1992)

There is a helpful result, due to **Date and Fagin (1992)**, that relates to conditions **detectable using FDs alone** — **completely ignoring JDs**:

"If a relation schema is in 3NF, and each of its keys consists of a single attribute, then it is also automatically in 5NF."

Why this is useful: This gives designers a **practical shortcut** — they don't need to hunt for elusive join dependencies at all in this specific case. If they can verify (using ordinary FD-based tests) that a relation is in **3NF** and has **only single-attribute candidate keys**, they can be confident it is **already in 5NF** as well, **without** ever having to test for JDs directly.

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and student-focused resources for every semester.

Connect With Us:

www.azdocuments.in [Click Here]

@azdocuments [Click Here]

Join Our Student Community: [Click Here]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.